计算机408大纲变化对比

科目	2024大纲	2025大纲
数据结构		一、基本概念【新增】 (一)数据结构的基本概念 (二)算法的基本概念
	三、树与二叉树 (四)树与二叉树的应用 1.哈夫曼(Hufnan)树和哈夫曼编码 2.并查集及其应用	四、树与二叉树 (四)树与二叉树的应用 1.哈夫曼(Hufman)树和哈夫曼编码 2.并查集及其应用 3.堆及其应用【新 增 】
计算机组成原理	无变化	
操作系统	二、进程管理 (一)进程与线程 5.进程间通信 共享内存，消息传递，管道。 (二)CPU调度与上下文切换 4.典型调度算法【 删 除 】 先来先服务调度算法，短作业(短进程短线程)优先调度算法，时间片轮转调度算	二、进程管理 (一)进程与线程 5.进程间通信 共享内存，消息传递，管道，信号【新 增 】 (二)CPU调度与上下文切换 4.CPU调度算法【新增】 5.多处理机调度【新增】
	三、内存管理 (二)虚拟内存管理 3.页框分配	三、内存管理 (二)虚拟存储管理 3.页框分配与回收【 新 增 】
计算机网络	无变化	

2-1 数据表示的相关基本概念

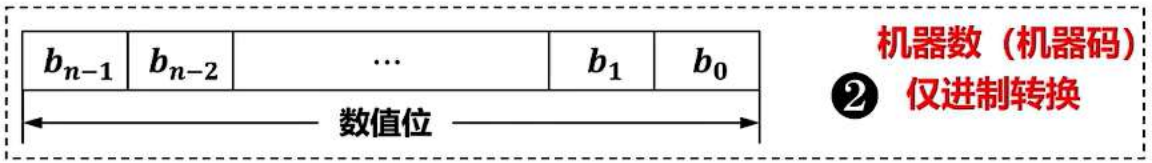


数据表示的相关基本概念

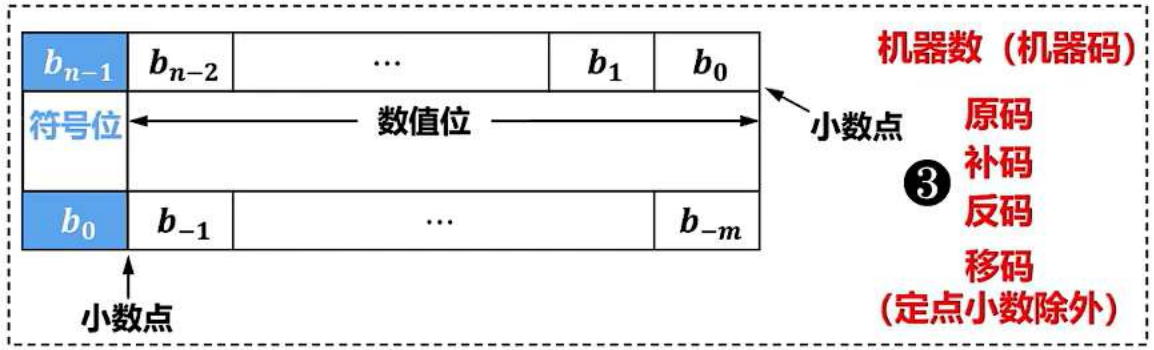
教学进程：本节课为❶，后续依次为❷ ❸ ❹ ❺

计算机内部参与运算的数
- 无符号数
- 有符号数
  - 定点数
    - 整数
    - 小数
  - 浮点数
    - 单精度
    - 双精度

❺ 对应C语言数据类型



机器数（机器码）
❷ 仅进制转换



机器数（机器码）
❸ 原码
补码
反码
移码
（定点小数除外）



机器数（机器码）
❹ IEEE 754 标准
阶码用移码
尾数用原码

⟺ 人们熟悉的数值
真值

2-2 进位计数制及其数据之间的相互转换



任意进制 ⟶ 十进制 (按“权”展开)

任意进制数 D 的取值为：

$$d_{n-1} \times r^{n-1} + d_{n-2} \times r^{n-2} + \cdots + d_1 \times r^1 + d_0 \times r^0 + d_{-1} \times r^{-1} + d_{-2} \times r^{-2} + \cdots + d_{-m} \times r^{-m}$$

	十进制	二进制	八进制	十六进制
数码d_i	$0,1,2,3,4,5,6,7,8,9$	$0,1$	$0,1,2,3,4,5,6,7$	$0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F$
基数r	10	2	8	16

【举例1】

$$(1011.101)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 11.625$$

$$(702.6)_8 = 7 \times 8^2 + 0 \times 8^1 + 2 \times 8^0 + 6 \times 8^{-1} = 450.75 = 7 \times (2^3)^2 + 0 \times (2^3)^1 + 2 \times (2^3)^0 + 6 \times (2^3)^{-1}$$

$$(A9DE.CF)_{16} = 10 \times 16^3 + 9 \times 16^2 + 13 \times 16^1 + 14 \times 16^0 + 12 \times 16^{-1} + 15 \times 16^{-2} = 43486.80859375$$

$$= 10 \times (2^4)^3 + 9 \times (2^4)^2 + 13 \times (2^4)^1 + 14 \times (2^4)^0 + 12 \times (2^4)^{-1} + 15 \times (2^4)^{-2}$$

任意进制 ⟶ 十进制 (按“权”展开)

【举例1】

$$(1011.101)_2 = 1\times2^3+0\times2^2+1\times2^1+1\times2^0+1\times2^{-1}+0\times2^{-2}+1\times2^{-3} = 11.625$$

$$(702.6)_8 = 7\times8^2+0\times8^1+2\times8^0+6\times8^{-1} = 450.75 = 7\times(2^3)^2+0\times(2^3)^1+2\times(2^3)^0+6\times(2^3)^{-1}$$

$$(A9DE.CF)_{16} = 10\times16^3+9\times16^2+13\times16^1+14\times16^0+12\times16^{-1}+15\times16^{-2} = 43486.80859375$$

$$= 10\times(2^4)^3+9\times(2^4)^2+13\times(2^4)^1+14\times(2^4)^0+12\times(2^4)^{-1}+15\times(2^4)^{-2}$$

常用2的幂次方数

2^{16}	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}
65536	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1	0.5	0.25	0.125

快速推导　　熟 记

任意进制 ⟶ 十进制 (按“权”展开)

【练习1】

$(1111)_2=(15)_{10}$

$(11110000)_2=(240)_{10}$

$(11111111)_2=(255)_{10}$

$(1100011001)_2=(793)_{10}$

$(10.101)_2=(2.625)_{10}$

$(1111)_8=(585)_{10}$

$(12340000)_8=(2736128)_{10}$

$(11111111)_8=(2396745)_{10}$

$(7654012345)_8=(1051727077)_{10}$

$(70.1)_8=(56.125)_{10}$

$(1111)_{16}=(4369)_{10}$

$(ABCD0000)_{16}=(2882338816)_{10}$

$(11111111)_{16}=(286331153)_{10}$

$(CFADEB)_{16}=(13610475)_{10}$

$(F0.1)_{16}=(240.0625)_{10}$

二进制 ⟷ 八进制 ， 二进制 ⟷ 十六进制

【练习2】

$(1111)_2=(17)_8$

$(11110000)_2=(360)_8$

$(1001101011)_2=(1153)_8$

$(10110.01)_2=(26.2)_8$

$(1111)_2=(F)_{16}$

$(11110000)_2=(F0)_{16}$

$(1001101011)_2=(26B)_{16}$

$(10110.01)_2=(16.4)_{16}$

$(127)_8=(57)_{16}$

$(361.72)_8=(F1.E8)_{16}$

$(69F)_{16}=(3237)_8$

$(59D.C9)_{16}=(2635.622)_8$

十进制 ⟶ 任意进制
(除基取余，乘基取整)

除基取余（针对整数部分） 低位先得

【举例3】

$(123)_{10} \rightarrow (1111011)_2$

2 | 123		
2 | 61	余数1	d_0
2 | 30	余数1	d_1
2 | 15	余数0	d_2
2 | 7	余数1	d_3
2 | 3	余数1	d_4
2 | 1	余数1	d_5
0	余数1	d_6

$(123)_{10} \rightarrow (173)_8$

8 | 123		
8 | 15	余数3	d_0
8 | 1	余数7	d_1
0	余数1	d_2

$(123)_{10} \rightarrow (7B)_{16}$

16 | 123		
16 | 7	余数B	d_0
0	余数7	d_1

十进制 ⟶ 任意进制
(除基取余，乘基取整)

乘基取整 （针对小数部分） 高位先得

【举例4】

$(0.6875)_{10} \rightarrow (0.1011)_2$

0.6875 × 2
1.375 整数 1 (d_{-1})
0.375 × 2
0.75 整数 0 (d_{-2})
0.75 × 2
1.5 整数 1 (d_{-3})
0.5 × 2
1.0 整数 1 (d_{-4})

$(0.6875)_{10} \rightarrow (0.54)_8$

0.6875 × 8
5.5 整数 5 (d_{-1})
0.5 × 8
4.0 整数 4 (d_{-2})

$(0.6875)_{10} \rightarrow (0.B)_{16}$

0.6875 × 16
11.0 整数 B (d_{-1})

十进制 ⟶ 任意进制
(除基取余，乘基取整)

整数部分 . 小数部分

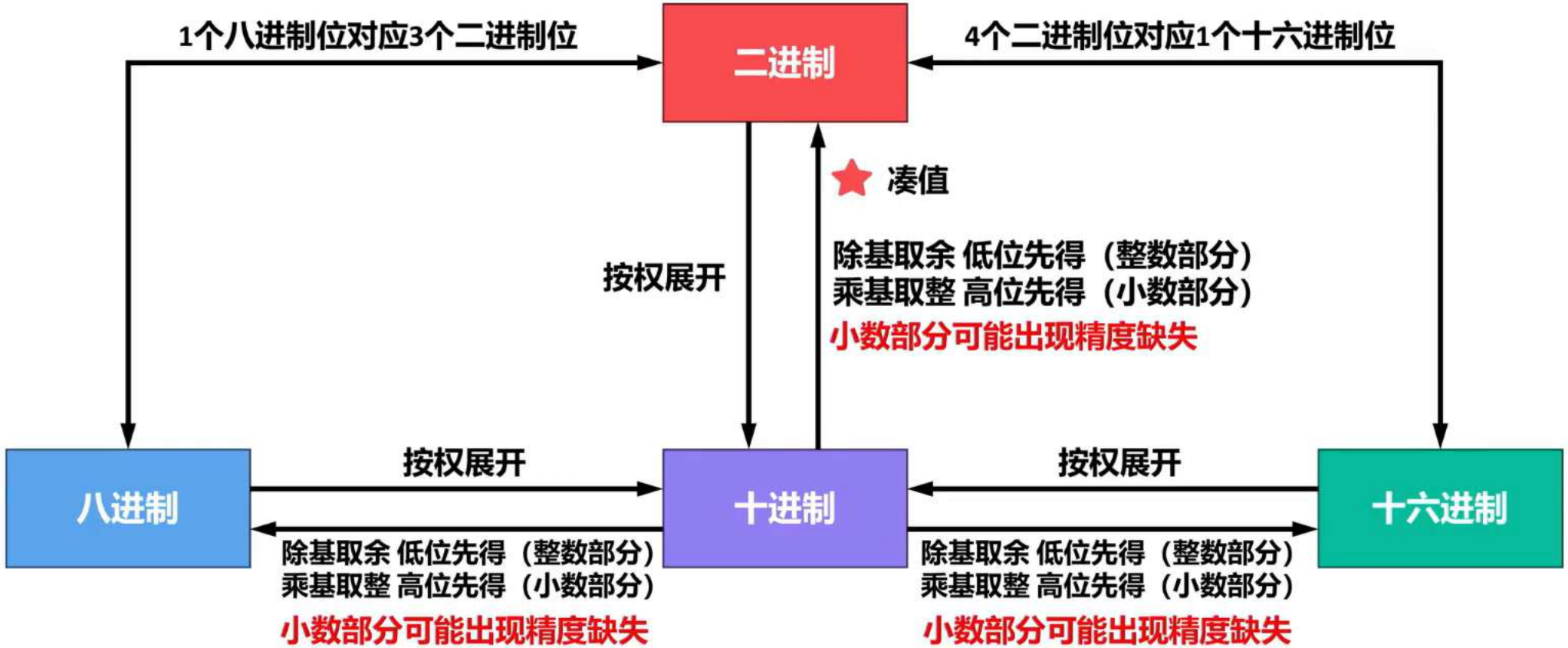
除基取余 低位先得　　**乘基取整 高位先得**

【练习3】

$$(516408.6)_{10} = (1111110000100111000.10011\ 0011\ 0011\cdots)_2$$

$$(516408.6)_{10} = (1760470.4\ 6314\ 6314\cdots)_8$$

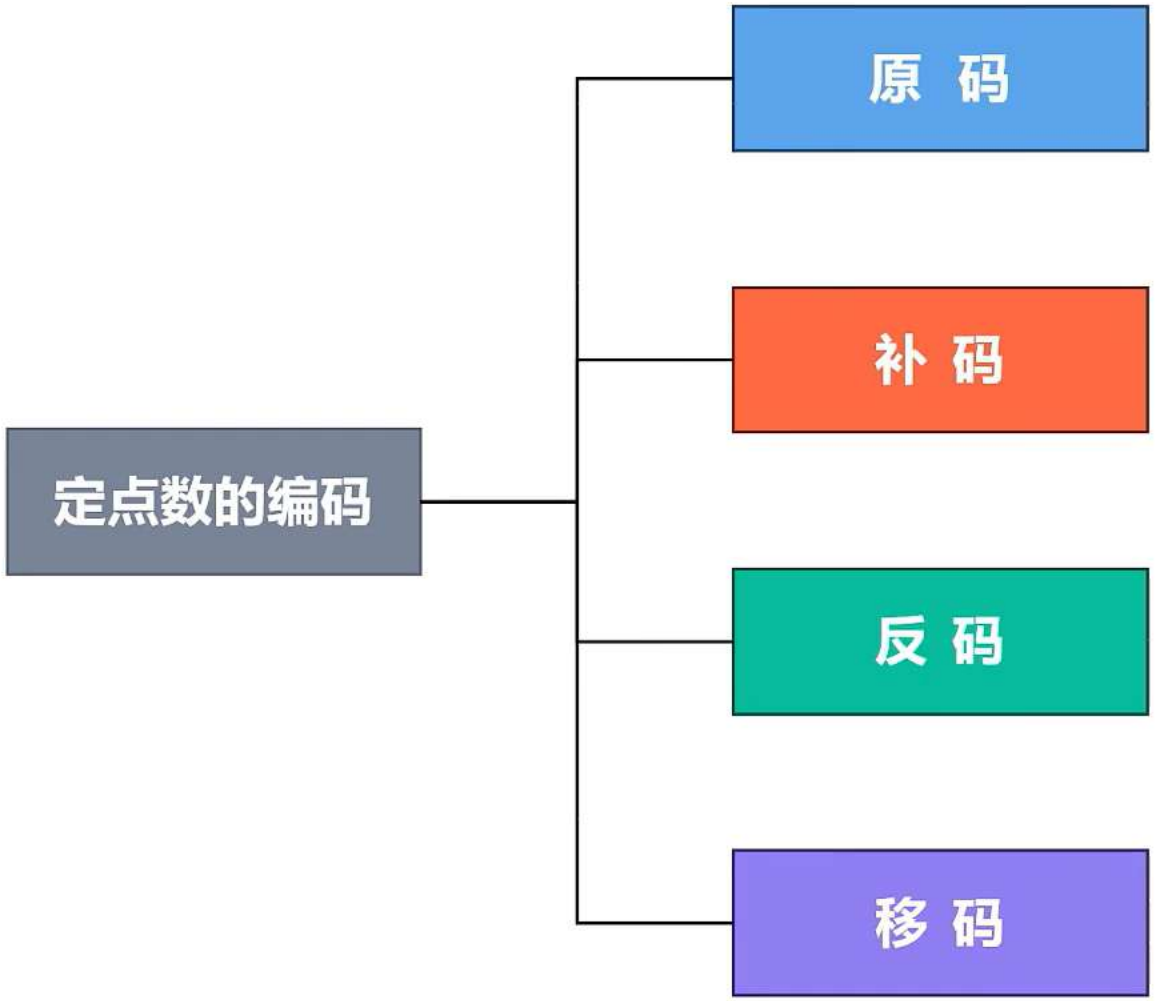
$$(516408.6)_{10} = (7E138.99\cdots)_{16}$$



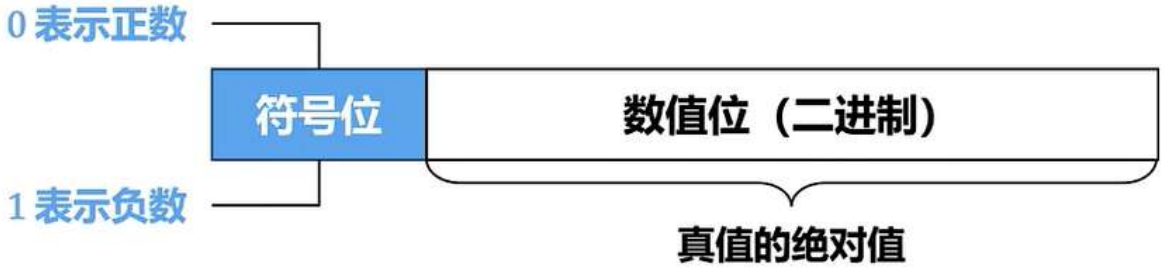
常用2的幂次方数

2^{16}	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^{9}	2^{8}	2^{7}	2^{6}	2^{5}	2^{4}	2^{3}	2^{2}	2^{1}	2^{0}	2^{-1}	2^{-2}	2^{-3}
65536	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1	0.5	0.25	0.125

快速推导　　熟 记


定点数的编码
原 码
补 码
反 码
移 码

原 码



原码表示又称为带符号的绝对值表示

优点

- 表示方法简单直观

缺点

- 真值0在原码中有两种不同的表示
- 符号位不能直接参与运算

原码在计算机中目前仅仅用于表示浮点数的尾码

定点整数的原码定义

假设真值x为定点整数，n为x的原码表示中数值位的位数（比特数量）。

定点小数的原码定义

假设真值x为定点小数

写出真值的原码
写出原码的真值

$$[x]_{原}=\begin{cases}0,x & 0\le x<2^n\\ 2^n+|x| & -2^n<x\le 0\end{cases}$$

$$[x]_{原}=\begin{cases}x & 0\le x<1\\ 1+|x| & -1<x\le 0\end{cases}$$

补码

■ 模的概念

- ☐ 模（或称模数）是一个数值计量系统的计量范围，记作mod或M。
- ☐ 只要确定了“模”，就可找到一个与负数等价的正数来代替此负数，该正数就是负数的补数。
- ☐ 超过计量范围的数都应该自动舍弃模数。

■ 补数的特点

- ☐ 一个负数可用它的正补数来替代，而这个正补数可以用模数加上负数本身求得。
- ☐ 一个正数和一个负数互为补数时，它们绝对值之和即为模数。
- ☐ 正数的补数即该正数本身。

优点

- 补码表示方法使得减法运算可以转换成加法运算。
- 真值0在补码中只有一种表示，这使得补码比原码多表示一个最小负数。
- 符号位可以直接参与运算，运算时符号位的进位作为模会被自动舍弃。

目前计算机中普遍采用补码表示有符号定点整数，例如C语言中char、short、int、long型整数都是采用补码进行表示的。

缺点

- 补码的表示相对原码更加复杂。
  - ☐ 原码的数值位与真值的绝对值相同。因此，通过原码可以很容易地得出真值。但是，补码就没有这么简单了。

定点整数的补码定义

假设真值x为定点整数，n为x的补码表示中数值位的位数（比特数量），加上1个符号位，x的补码表示共有$n+1$位，最低位的位权值为2^0，而最高位（符号位）的位权值为2^n，因此最高位进位的位权值为2^{n+1}，即模数为2^{n+1}。

定点小数的补码定义 通常不会涉及定点小数的补码表示

假设真值x为定点小数（纯小数），小数点左侧的位为最高位（符号位），其位权值为2^0，而最高位进位的位权值为2^1，即模数为$2^1=2$。

写出真值的补码
写出补码的真值

$$[x]_{补}=\begin{cases}0,x & 0\le x<2^n\\ 2^{n+1}+x & -2^n\le x<0 \pmod{2^{n+1}}\end{cases}$$

$$[x]_{补}=\begin{cases}x & 0\le x<1\\ 2+x & -1\le x<0 \pmod{2}\end{cases}$$

反码

■ 反码通常用来作为由原码求补码或者由补码求原码的中间过渡。

- □ 正数的反码：符号位为0，数值位就是它本身。
- □ 负数的反码：符号位为1，数值位就是真值数值位取反。

优 点

- 符号位可以参与运算。

缺点

- 最高位（符号位）产生的进位要加到运算结果的低位（循环进位）
- 真值0在反码中有两种不同的表示

反码在计算机中目前很少被使用

定点整数的反码定义

假设真值x为定点整数， n为x的反码表示中数值位的位数（比特数量）。

$$[x]_{反}=\begin{cases}0,x & 0\le x<2^n\\ (2^{n+1}-1)+x & -2^n<x\le 0\end{cases}$$

定点小数的反码定义

假设真值x为定点小数， n为x的反码表示中数值位的位数（比特数量）。

写出真值的反码
写出反码的真值

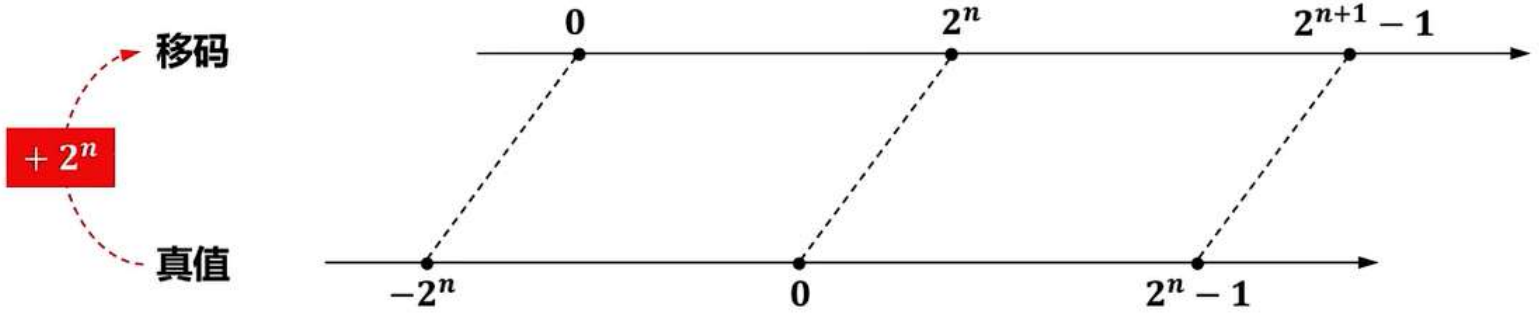
$$[x]_{反}=\begin{cases}x & 0\le x<1\\ (2-2^{-n})+x & -1<x\le 0\end{cases}$$

移码

■ 移码就是在真值上加一个常数 2^n 。

□ 在数轴上，移码所表示的范围对应于真值在数轴上的范围向轴的正方向移动2^n个单元。

□ 移码只用于定点整数的表示。



移码定义

假设真值x为定点整数， n为x的移码表示中数值位的位数（比特数量）。

$$[x]_{移} = x + 2^n \qquad -2^n \le x < 2^n$$

优 点

- 真值0在移码中只有一种表示。
- 移码保持了真值原有的大小顺序，可以直接比较大小。
- 最小真值的移码为全0，最大真值的移码为全1，符合人们的习惯。

当浮点数的阶码用移码来表示时，就能很方便地比较阶码的大小。

真值x为正数

真值x ←符号位为0→ $[x]_{原}$ ←相同→ $[x]_{反}$ ←相同→ $[x]_{补}$ ←符号位相反→ $[x]_{移}$

真值x为负数

扫描法 （非常适合手工计算求补码）

仅对数值位从右到左顺序扫描，右起第一个1及其右边的0保持不变，其余各位取反

反码法 （非常适合利用硬件求补码）

真值x ←符号位为1→ $[x]_{原}$ ←数值位按位取反→ $[x]_{反}$ ⇢末位加1⇢ $[x]_{补}$ ←符号位相反→ $[x]_{移}$

全部位按位取反，末位加1 → $[-x]_{补}$

补码的补码是原码（原码与补码互为补码）



已知原码、反码、补码，如何算出真值（十进制形式）？

- 正数：将原码、反码、补码的数值位按权展开相加，符号位的0表示“+”
- 负数：将反码、补码转换成原码，然后将原码的数值位按权展开相加，符号位的1表示“−”

2-4-1 浮点数的表示

—— 浮点数的表示形式和表示范围



定点数表示

	机器字长为$n+1$位（1个符号位，n个数值位）						
	定点小数表示范围			定点整数表示范围			
	原 码	反 码	补 码	原 码	反 码	补 码	移 码
最大正数	$0.111\cdots11$ $(1-2^{-n})$	$0.111\cdots11$ $(1-2^{-n})$	$0.111\cdots11$ $(1-2^{-n})$	$0,111\cdots11$ (2^n-1)	$0,111\cdots11$ (2^n-1)	$0,111\cdots11$ (2^n-1)	$1,111\cdots11$ (2^n-1)
最小正数	$0.000\cdots01$ (2^{-n})	$0.000\cdots01$ (2^{-n})	$0.000\cdots01$ (2^{-n})	$0,000\cdots01$ (1)	$0,000\cdots01$ (1)	$0,000\cdots01$ (1)	$1,000\cdots01$ (1)
0	$0.000\cdots00$ $1.000\cdots00$	$0.000\cdots00$ $1.111\cdots11$	$0.000\cdots00$	$0,000\cdots00$ $1,000\cdots00$	$0,000\cdots00$ $1,111\cdots11$	$0,000\cdots00$	$1,000\cdots00$
最大负数	$1.000\cdots01$ (-2^{-n})	$1.111\cdots10$ (-2^{-n})	$1.111\cdots11$ (-2^{-n})	$1,000\cdots01$ (-1)	$1,111\cdots10$ (-1)	$1,111\cdots11$ (-1)	$0,111\cdots11$ (-1)
最小负数	$1.111\cdots11$ $-(1-2^{-n})$	$1.000\cdots00$ $-(1-2^{-n})$	$1.000\cdots00$ (-1)	$1,111\cdots11$ $-(2^n-1)$	$1,000\cdots00$ $-(2^n-1)$	$1,000\cdots00$ (-2^n)	$0,000\cdots00$ (-2^n)

- 定点数所能表示的数据范围与下列因素有关。
  - 机器字长：字长越长，其表示的数据范围就越大。
  - 所采用的机器码：补码和移码所能表示的数据范围，比原码和反码所能表示的数据范围要多一个最小负数。

浮点数表示

【举例】

二进制浮点数 $110.0101 = 2^{10} \times 1.100101$

$= 2^{-1} \times 1100.101$

$= 2^{11} \times 0.1100101$

可取2、4、8、16等

隐含约定 基数r

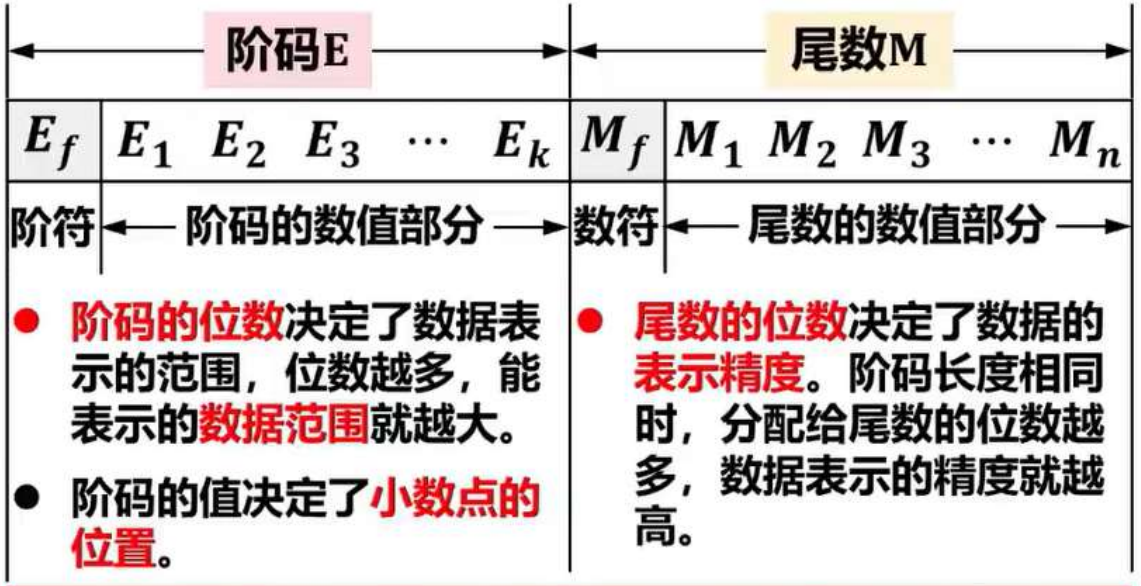
定点整数 阶码$E(Exponent)$

定点小数 尾数$M(Mantissa)$

任意一个二进制数N可表示成如下形式：

$$N = r^E \times M$$

浮点数在机器中的表示形式



- 阶码的位数决定了数据表示的范围，位数越多，能表示的数据范围就越大。
- 阶码的值决定了小数点的位置。
- 尾数的位数决定了数据的表示精度。阶码长度相同时，分配给尾数的位数越多，数据表示的精度就越高。

- 阶码可采用原码、补码、反码、移码进行表示。
- 尾数可采用原码、补码、反码进行表示。
- 对应的浮点数表示范围会略有不同。

浮点数表示范围

浮点数在机器中的表示形式

$N = r^E \times M$ (r可取2、4、8、16等)

正数N最大值

正数N最小值（浮点数的最小精度）

负数N最小值

负数N最大值

【举例】

正数N最大值 $2^{+3} \times (+0.75)$

正数N最小值 $2^{-3} \times (+0.25)$

负数N最小值 $2^{+3} \times (-0.75)$

负数N最大值 $2^{+3} \times (-0.25)$

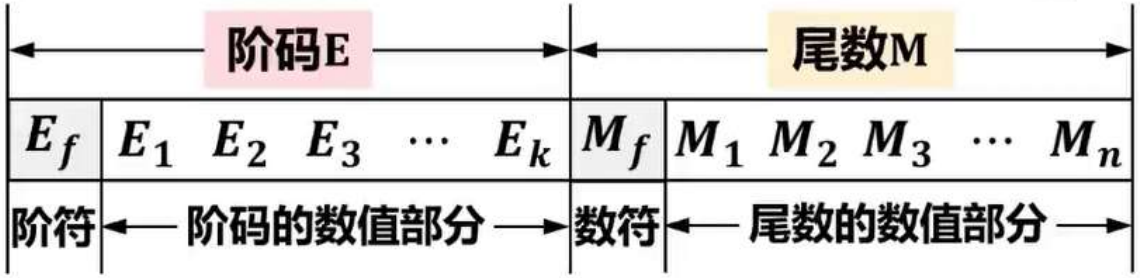
阶码E						尾数M					
E_f	E_1	E_2	E_3	…	E_k	M_f	M_1	M_2	M_3	…	M_n
阶符	阶码的数值部分					数符	尾数的数值部分				
最大值						正数最大值					
最小值						正数最小值					
最大值						负数最小值					
最小值						负数最大值					
3位阶码（原码表示） （1个阶符，2个数值位）						3位尾数（原码表示） （1个数符，2个数值位）					
最大值0,11，即+3						正数最大值0.11，即+0.75					
最小值1,11，即−3						正数最小值0.01，即+0.25					
最大值0,11，即+3						负数最小值1.11，即−0.75					
最小值1,11，即−3						负数最大值1.01，即−0.25					

浮点数表示范围

- 尽管浮点数有效扩大了数据表示范围，但受机器字长限制，浮点数仍然存在溢出现象。
  - 当浮点数的阶码大于最大阶码时，称为上溢，此时机器停止运算，浮点运算器件会显示溢出标志。
  - 当浮点数的阶码小于最小阶码时，称为下溢，虽然此时数据不能被精确表示，但由于发生下溢时数据的绝对值很小，通常将尾数各位强置为0，按机器0处理，此时机器可以继续运行。
- 当一个浮点数在正、负数区域中但并不在某个数轴刻度上时，也会出现精度溢出的问题，此时只能用近似数表示。

浮点数在机器中的表示形式

$N = r^E \times M$ (r可取2、4、8、16等)

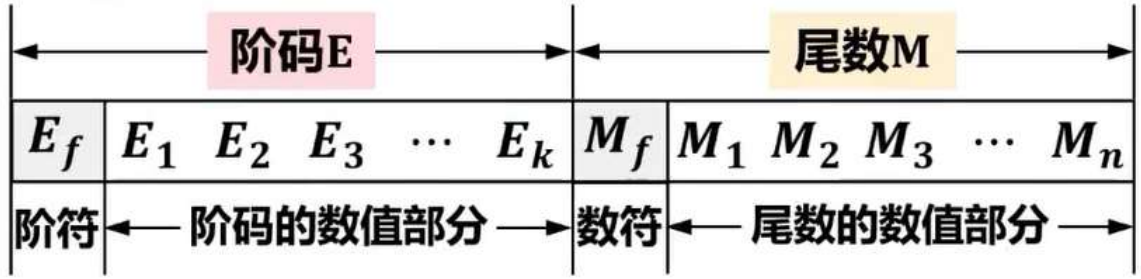




浮点数表示范围

浮点数在机器中的表示形式

$N = r^E \times M$ (r可取2、4、8、16等)





■ 一旦浮点数的位数确定后，合理分配阶码E和尾数M的位数，直接影响浮点数的表示范围和精度。

- □ 短实数（32位）：阶码E取8位（含1位阶符），尾数M取24位（含1位数符）。
- □ 长实数（64位）：阶码E取11位（含1位阶符），尾数M取53位（含1位数符）。 IEEE 754
- □ 临时实数（80位）：阶码E取15位（含1位阶符），尾数M取65位（含1位数符）。

2-4-2 浮点数的表示

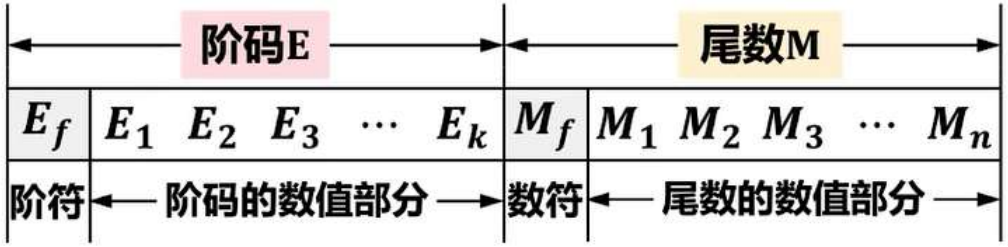
—— 浮点数的规格化

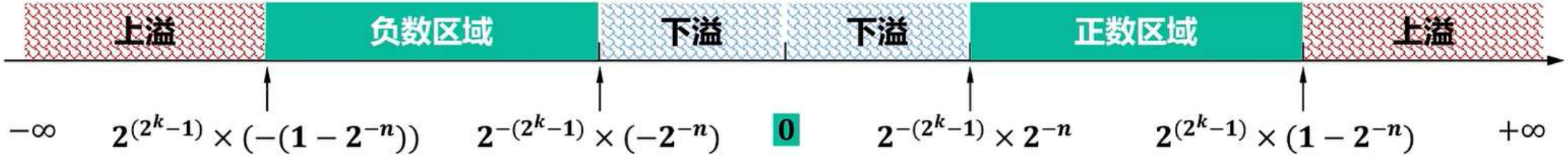


浮点数规格化后的表示范围

浮点数在机器中的表示形式

$N = r^E \times M$ (r可取2、4、8、16等)





非规格化

$2^{(2^k-1)} \times (-(1-2^{-n}))$ 最小负数

$2^{-(2^k-1)} \times (-2^{-n})$ 最大负数

$2^{-(2^k-1)} \times 2^{-n}$ 最小正数

$2^{(2^k-1)} \times (1-2^{-n})$ 最大正数

规格化

$2^{(2^k-1)} \times (-(1-2^{-n}))$ 最小负数

$2^{-(2^k-1)} \times (-2^{-1})$ 最大负数

$2^{-(2^k-1)} \times 2^{-1}$ 最小正数

$2^{(2^k-1)} \times (1-2^{-n})$ 最大正数

规格化后，尾数的数值位的最高位为1，尾数的其余位为0

浮点数的规格化

- 通常要求浮点数在数据表示时对尾数进行规格化处理，即使得尾数的最高数值位必须是一个有效值。
- 浮点数规格化带来以下好处：
  - 使浮点数的表示形式唯一。
  - 使浮点数的表示精度最高。
- 对于基数r不同的浮点数，因其规格化数的形式不同，规格化过程也不同。
  - 当$r=2$时，尾数数值部分最高位为1的数为规格化数。
    - 左规：尾数数值部分每左移1位，阶码减1。
    - 右规：尾数数值部分每右移1位，阶码加1。
  - 当$r=4$时，尾数数值部分最高2位不全为0的数为规格化数。
    - 左规：尾数数值部分每左移2位，阶码减1。
    - 右规：尾数数值部分每右移2位，阶码加1。
  - 当$r=8$时，尾数数值部分最高3位不全为0的数为规格化数。
    - 左规：尾数数值部分每左移3位，阶码减1。
    - 右规：尾数数值部分每右移3位，阶码加1。

依此类推，不难得到基数r为16或2^n时的规格化过程。

- 基数r不同，对数的表示范围和精度等都有影响。一般来说：
  - 基数r越大，可表示的浮点数范围越大，而且所表示的数的个数越多。但浮点数的精度反而下降。

2-4-3 浮点数的表示

—— IEEE 754浮点数标准



IEEE 754 浮点数标准

- **阶码：定点整数，用移码表示。** 复习一下之前学习过的移码

移码定义 假设真值x为定点整数，n为x的移码表示中数值位的位数（比特数量）。

$$[x]_{移} = x + 2^n \qquad -2^n \le x < 2^n$$



$$[x]_{移} = x + 2^3 \qquad -2^3 \le x < 2^3$$

$$[0]_{移} = 0 + 2^3 = 8 = 1000$$

$$[1]_{移} = 1 + 2^3 = 9 = 1001$$

$$[2]_{移} = 2 + 2^3 = 10 = 1010$$

$$[7]_{移} = 7 + 2^3 = 15 = 1111$$

$$[-1]_{移} = -1 + 2^3 = 7 = 0111$$

$$[-2]_{移} = -2 + 2^3 = 6 = 0110$$

$$[-8]_{移} = -8 + 2^3 = 0 = 0000$$

IEEE 754 浮点数标准

- **阶码：定点整数，用移码表示。** 复习一下之前学习过的移码

移码定义 假设真值x为定点整数，n为x的移码表示中数值位的位数（比特数量）。

$$[x]_{移} = x + 2^n \qquad -2^n \le x < 2^n$$



不考虑移码的符号位，看作无符号二进制数

优点

- 真值0在移码中只有一种表示。
- 移码保持了真值原有的大小顺序，可以直接比较大小。
- 最小真值的移码为全0，最大真值的移码为全1，符合人们的习惯。

当浮点数的阶码用移码来表示时，就能很方便地比较阶码的大小。

IEEE 754 浮点数标准

- **阶码：** 定点整数，用移码表示。 **复习一下之前学习过的移码**

移码定义 假设真值x为定点整数，n为x的移码表示中数值位的位数（比特数量）。

$$[x]_{移} = x + 2^n \qquad -2^n \le x < 2^n$$

$$[x]_{移} = x + 2^7 \qquad -2^7 \le x < 2^7$$



其中：

- **符号：** 取值0表示正数；取值1表示负数。
- **阶码：** 定点整数，用移码表示。
- **尾数：** 定点小数，用原码表示。

IEEE 754 浮点数标准

- **阶码：定点整数，用移码表示。** 复习一下之前学习过的移码

移码定义 假设真值x为定点整数，n为x的移码表示中数值位的位数（比特数量）。

$$[x]_{移} = x + 2^n \qquad -2^n \le x < 2^n$$

$$[x]_{移} = x + 2^7 \qquad -2^7 \le x < 2^7$$

■ 在IEEE 754 浮点数标准中，32位单精度浮点数的8位阶码尽管采用移码表示，但采用偏移常数是$2^7 - 1 = 127$，而不是标准移码的$2^7 = 128$。

$$[x]_{移} = x + (2^7 - 1) \qquad -2^7 \le x < 2^7$$

$[x]_{移}$	1,1111111	0,0000000	0,0000001	0,1111101	0,1111110	0,1111111	1,0000000	1,0000001	1,1111100	1,1111101	1,1111110
$[x]_{移}$	0,0000000	0,0000001	0,0000010	0,1111110	0,1111111	1,0000000	1,0000001	1,0000010	1,1111101	1,1111110	1,1111111
真值x	-128	-127	-126	… -2	-1	0	1	2	… 125	126	127

IEEE 754 浮点数标准

■ 在IEEE 754 浮点数标准中，32位单精度浮点数的8位阶码尽管采用移码表示，但采用偏移常数是$2^7-1=127$，而不是标准移码的$2^7=128$。

$$[x]_{移}=x+(2^7-1) \qquad -2^7\le x<2^7$$

为什么偏移常数不采用标准的128，而采用127？

采用偏移常数128表示的最小规格化数的倒数会发生溢出，而采用偏移常数127表示的任何一个规格化数的倒数则不会溢出。



阶码为全1、全0有特殊用途

$[x]_{移}$	1,1111111	0,0000000	0,0000001	0,1111101	0,1111110	0,1111111	1,0000000	1,0000001	1,1111100	1,1111101	1,1111110
$[x]_{移}$	0,0000000	0,0000001	0,0000010	0,1111110	0,1111111	1,0000000	1,0000001	1,0000010	1,1111101	1,1111110	1,1111111
真值x	-128	-127	-126	… -2	-1	0	1	2	… 125	126	127

IEEE 754 浮点数标准

为什么偏移常数不采用标准的128，而采用127？

采用偏移常数128表示的最小规格化数的倒数会发生溢出，而采用偏移常数127表示的任何一个规格化数的倒数则不会溢出。



阶码为全1、全0有特殊用途

$[x]_{移}$	1,1111111	0,0000000	0,0000001	0,1111101	0,1111110	0,1111111	1,0000000	1,0000001	1,1111100	1,1111101	1,1111110
$[x]_{移}$	0,0000000	0,0000001	0,0000010	0,1111110	0,1111111	1,0000000	1,0000001	1,0000010	1,1111101	1,1111110	1,1111111
真值x	-128	-127	-126	… -2	-1	0	1	2	… 125	126	127

为简单起见，仅从8位阶码考虑，而不考虑23位尾数

采用偏移常数128表示的最小规格化数的阶码，其取值应为能取到的最小值-128。

采用偏移常数127表示的最小规格化数的阶码，其取值应为能取到的最小值-126。

$$2^{-128} \xrightarrow{\text{倒数}} \frac{1}{2^{-128}} = 2^{128}$$

超越最大值而溢出

$$2^{-126} \xrightarrow{\text{倒数}} \frac{1}{2^{-126}} = 2^{126}$$

在-126~127之间，未溢出

IEEE 754 浮点数标准

■ IEEE 754 标准主要包括两种基本的浮点数格式：

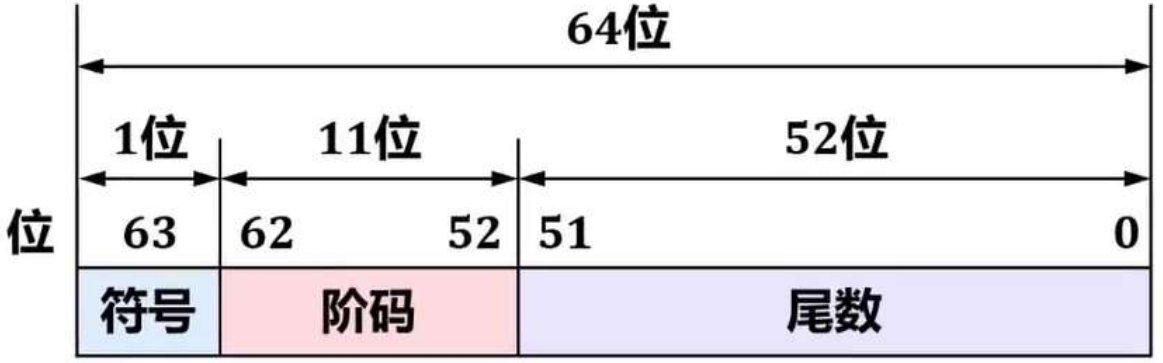
□ 32位单精度浮点数

对应C语言中的float型。



□ 64位双精度浮点数

对应C语言中的double型。

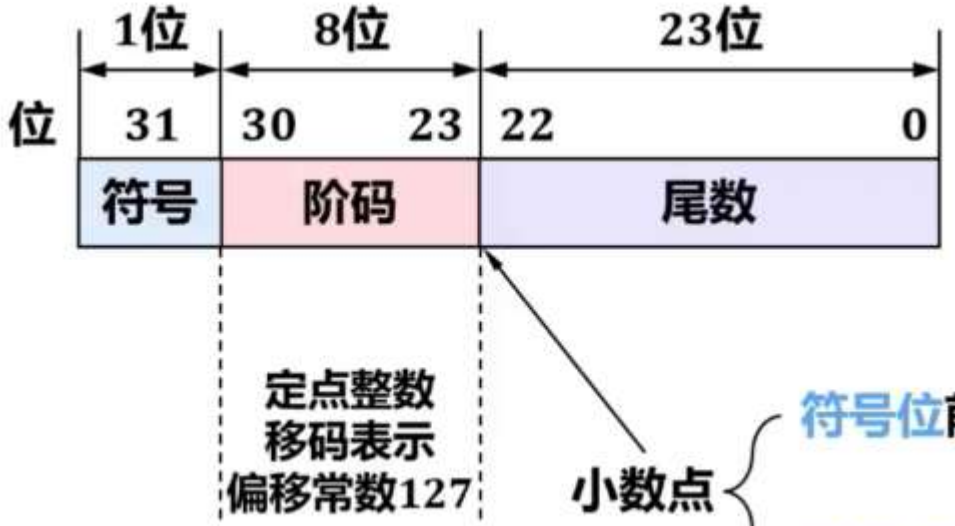


其中：

- 符号：取值0表示正数；取值1表示负数。
- 阶码：定点整数，用移码表示。
- 尾数：定点小数，用原码表示。

IEEE 754 浮点数标准

■ 下面以32位单精度浮点数为例介绍IEEE 754单精度浮点数标准。

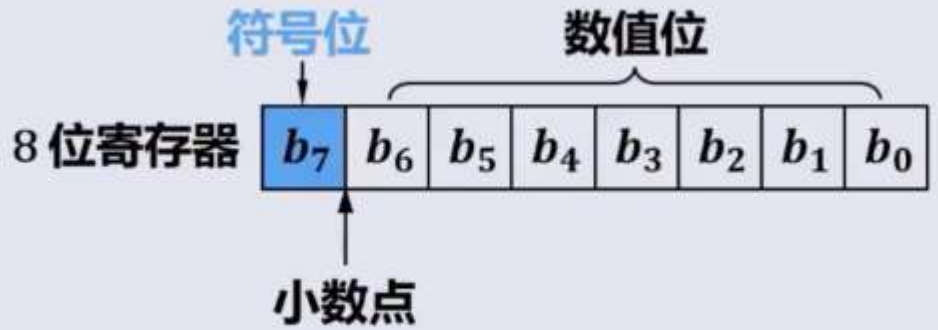


- 符号：取值0表示正数；取值1表示负数。
- 阶码：定点整数，用移码表示，偏置常数$2^7-1=127$。
- 尾数：定点小数，用原码表示。

小数点
- 符号位前移到最左侧。
- 相邻左侧隐藏一个1，表示数值而不表示符号。尾数实际有24位，但不保存隐藏的那个1，只保存23位，节省的比特位可用于提高尾数的精度。

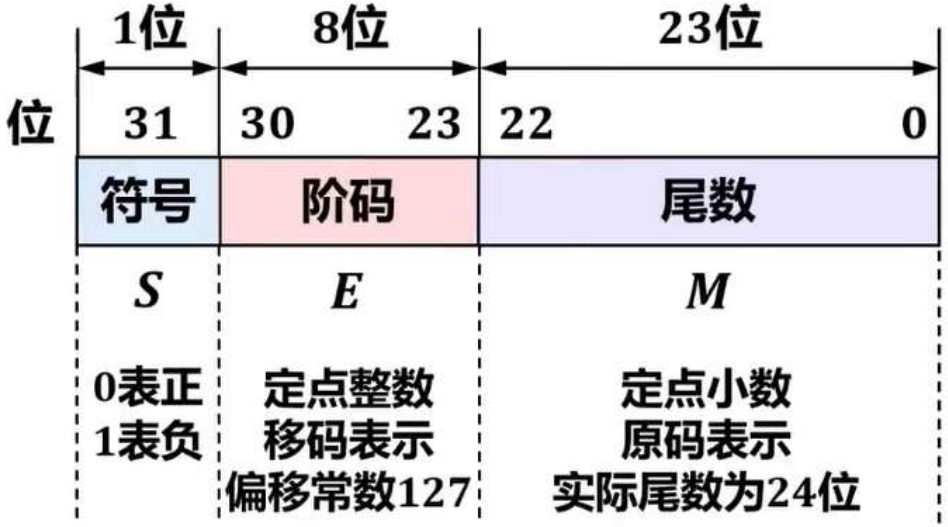
复习

定点小数格式举例



IEEE 754 浮点数标准

■ 下面以32位单精度浮点数为例介绍IEEE 754单精度浮点数标准。



- 符号：取值0表示正数；取值1表示负数。
- 阶码：定点整数，用移码表示，偏置常数$2^7 - 1 = 127$。
- 尾数：定点小数，用原码表示。符号位前移到最左侧。

  相邻左侧隐藏一个1，表示数值而不表示符号。
  尾数实际有24位，但不保存隐藏的那个1，只保存23位，节省的比特位可用于提高尾数的精度。
  完整的尾数形式为$1.M$



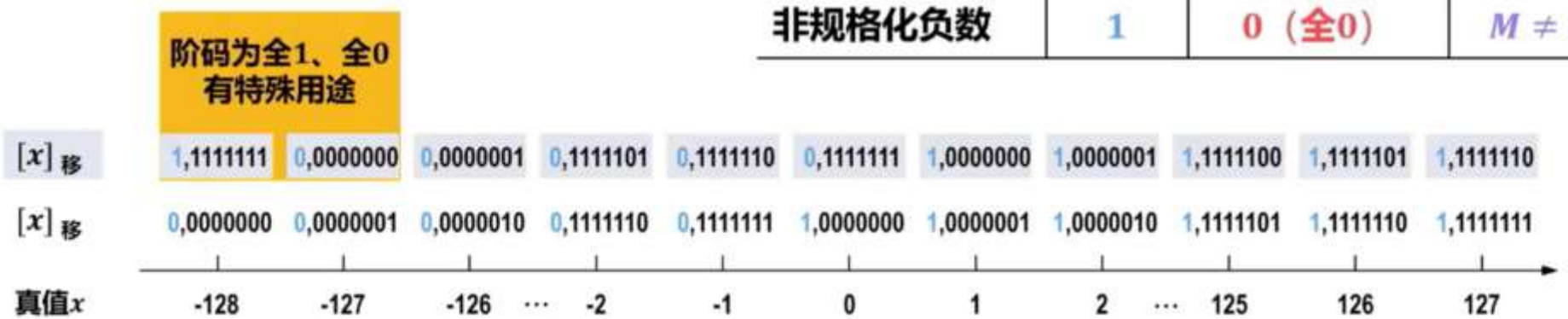
假设x为阶码E的真值，即 $E=[x]_{移}$

$[x]_{移}=e+127$

$E=x+127 \longrightarrow x=E-127$

IEEE 754 单精度（32位）浮点数标准

数值的分类	符号S	阶码E	尾数M	真值
正零	0	0（全0）	0	$+0$
负零	1	0（全0）	0	-0
正无穷大	0	255（全1）	0	$+\infty$
负无穷大	1	255（全1）	0	$-\infty$
无定义数（非数）	0或1	255（全1）	$\neq 0$	$NaN(not\ a\ number)$
规格化正数	0	$1 \le E \le 254$	M	$(-1)^0 \times 2^{E-127} \times 1.M$
规格化负数	1	$1 \le E \le 254$	M	$(-1)^1 \times 2^{E-127} \times 1.M$
非规格化正数	0	0（全0）	$M \neq 0$	$(-1)^0 \times 2^{-126} \times 0.M$
非规格化负数	1	0（全0）	$M \neq 0$	$(-1)^1 \times 2^{-126} \times 0.M$



IEEE 754 单精度（32位）浮点数标准

	数值的分类	符号S	阶码E	尾数M	真值	
机器0	正零	0	0（全0）	0	$+0$	阶码E为全0
机器0	负零	1	0（全0）	0	-0	阶码E为全0
非规格化数	非规格化正数	0	0（全0）	$M \neq 0$	$(-1)^0 \times 2^{-126} \times 0.M$	阶码E为全0
非规格化数	非规格化负数	1	0（全0）	$M \neq 0$	$(-1)^1 \times 2^{-126} \times 0.M$	阶码E为全0
无穷大	正无穷大	0	255（全1）	0	$+\infty$	阶码E为全1
无穷大	负无穷大	1	255（全1）	0	$-\infty$	阶码E为全1
非数NaN	无定义数（非数）	0或1	255（全1）	$\neq 0$	$NaN(not\ a\ number)$	阶码E为全1
规格化数	规格化正数	0	$1 \leq E \leq 254$	M	$(-1)^0 \times 2^{E-127} \times 1.M$	阶码E非全0和全1
规格化数	规格化负数	1	$1 \leq E \leq 254$	M	$(-1)^1 \times 2^{E-127} \times 1.M$	阶码E非全0和全1

- 一般情况下，$+0$和-0是等效的。
- 非规格化数可用于处理阶码下溢，使得出现比最小规格化数还小的数时程序也能继续进行下去。
- 引入无穷大数可使计算过程出现异常的情况下程序能继续执行，并且可为程序提供错误检测功能。例如非0浮点数除0运算的结果就是无穷大，因此非0浮点数除0不会像整型数除0一样产生严重错误。
- 非数NaN则用于表示$\frac{0}{0}$、$\frac{\infty}{\infty}$、$0 \times \infty$、负数的平方根等。部分非数NaN运算结果可能会产生异常。

IEEE 754 浮点数标准

数值的分类	单精度（32位，对应C语言中的float）				双精度（64位，对应C语言中的double）			
	符号S	阶码E	尾数M	真值	符号S	阶码E	尾数M	真值
正零	0	0（全0）	0	$+0$	0	0（全0）	0	$+0$
负零	1	0（全0）	0	-0	1	0（全0）	0	-0
非规格化正数	0	0（全0）	$M \neq 0$	$(-1)^0 \times 2^{-126} \times 0.M$	0	0（全0）	$M \neq 0$	$(-1)^0 \times 2^{-1022} \times 0.M$
非规格化负数	1	0（全0）	$M \neq 0$	$(-1)^1 \times 2^{-126} \times 0.M$	1	0（全0）	$M \neq 0$	$(-1)^1 \times 2^{-1022} \times 0.M$
正无穷大	0	255（全1）	0	$+\infty$	0	2047（全1）	0	$+\infty$
负无穷大	1	255（全1）	0	$-\infty$	1	2047（全1）	0	$-\infty$
无定义数（非数）	0或1	255（全1）	$\neq 0$	$NaN(not\ a\ number)$	0或1	2047（全1）	$\neq 0$	$NaN(not\ a\ number)$
规格化正数	0	$1 \leq E \leq 254$	M	$(-1)^0 \times 2^{E-127} \times 1.M$	0	$1 \leq E \leq 2046$	M	$(-1)^0 \times 2^{E-1023} \times 1.M$
规格化负数	1	$1 \leq E \leq 254$	M	$(-1)^1 \times 2^{E-127} \times 1.M$	1	$1 \leq E \leq 2046$	M	$(-1)^1 \times 2^{E-1023} \times 1.M$

- 一般情况下，+0和−0是等效的。
- 非规格化数可用于处理阶码下溢，使得出现比最小规格化数还小的数时程序也能继续进行下去。
- 引入无穷大数可使计算过程出现异常的情况下程序能继续执行，并且可为程序提供错误检测功能。例如非0浮点数除0运算的结果就是无穷大，因此非0浮点数除0不会像整型数除0一样产生严重错误。
- 非数NaN则用于表示$\frac{0}{0}$、$\frac{\infty}{\infty}$、$0 \times \infty$、负数的平方根等。部分非数NaN运算结果可能会产生异常。

IEEE 754 单精度（32位）浮点数表示范围

数值的分类	符号S	阶码E	尾数M	真值
规格化正数	0	$1 \le E \le 254$	M	$(-1)^0 \times 2^{E-127} \times 1.M$
规格化负数	1	$1 \le E \le 254$	M	$(-1)^1 \times 2^{E-127} \times 1.M$
非规格化正数	0	0（全0）	$M \neq 0$	$(-1)^0 \times 2^{-126} \times 0.M$
非规格化负数	1	0（全0）	$M \neq 0$	$(-1)^1 \times 2^{-126} \times 0.M$

$1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + \cdots + 1 \times 2^{-23}$ 等比数列求和

规格化正数最大值 $(-1)^0 \times 2^{254-127} \times 1.111\cdots11 = 2^{254-127} \times (2 - 2^{-23}) \approx 3.4 \times 10^{38}$

规格化正数最小值 $(-1)^0 \times 2^{1-127} \times 1.0 = 2^{-126}$

规格化负数最大值 $(-1)^1 \times 2^{1-127} \times 1.0 = -2^{-126}$

规格化负数最小值 $(-1)^1 \times 2^{254-127} \times 1.111\cdots11 = -(2^{254-127} \times (2 - 2^{-23})) \approx -3.4 \times 10^{38}$

IEEE 754 单精度（32位）浮点数表示范围

数值的分类	符号S	阶码E	尾数M	真值
规格化正数	0	$1 \le E \le 254$	M	$(-1)^0 \times 2^{E-127} \times 1.M$
规格化负数	1	$1 \le E \le 254$	M	$(-1)^1 \times 2^{E-127} \times 1.M$
非规格化正数	0	0（全0）	$M \neq 0$	$(-1)^0 \times 2^{-126} \times 0.M$
非规格化负数	1	0（全0）	$M \neq 0$	$(-1)^1 \times 2^{-126} \times 0.M$

非规格化正数最大值 $(-1)^0 \times 2^{-126} \times 0.111\cdots11 = 2^{-126} \times (1 - 2^{-23})$

非规格化正数最小值 $(-1)^0 \times 2^{-126} \times 0.000\cdots01 = 2^{-126} \times 2^{-23} = 2^{-149}$

非规格化负数最大值 $(-1)^1 \times 2^{-126} \times 0.000\cdots01 = 2^{-126} \times 2^{-23} = -2^{-149}$

非规格化负数最小值 $(-1)^1 \times 2^{-126} \times 0.111\cdots11 = -2^{-126} \times (1 - 2^{-23})$

IEEE 754 浮点数表示范围

数值的分类			单精度（32位，对应C语言中的float）	双精度（64位，对应C语言中的double）
规格化	正数	最大值	$(-1)^0\times 2^{254-127}\times 1.111\cdots 1=2^{127}\times\left(2-2^{-23}\right)$ $\approx 3.4\times 10^{38}$	$(-1)^0\times 2^{2046-1023}\times 1.111\cdots 1=2^{1023}\times\left(2-2^{-52}\right)$ $\approx 1.8\times 10^{308}$
		最小值	$(-1)^0\times 2^{1-127}\times 1.0=2^{-126}$	$(-1)^0\times 2^{1-1023}\times 1.0=2^{-1022}$
	负数	最大值	$(-1)^1\times 2^{1-127}\times 1.0=-2^{-126}$	$(-1)^1\times 2^{1-1023}\times 1.0=-2^{-1022}$
		最小值	$(-1)^1\times 2^{254-127}\times 1.111\cdots 1=-(2^{127}\times(2-2^{-23}))$ $\approx -3.4\times 10^{38}$	$(-1)^1\times 2^{2046-1023}\times 1.111\cdots 1=-(2^{1023}\times(2-2^{-52}))$ $\approx -1.8\times 10^{308}$
非规格化	正数	最大值	$(-1)^0\times 2^{-126}\times 0.111\cdots 11=2^{-126}\times(1-2^{-23})$	$(-1)^0\times 2^{-1022}\times 0.111\cdots 11=2^{-1022}\times(1-2^{-52})$
		最小值	$(-1)^0\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=2^{-149}$	$(-1)^0\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=2^{-1074}$
	负数	最大值	$(-1)^1\times 2^{-126}\times 0.000\cdots 01=2^{-126}\times 2^{-23}=-2^{-149}$	$(-1)^1\times 2^{-1022}\times 0.000\cdots 01=2^{-1022}\times 2^{-52}=-2^{-1074}$
		最小值	$(-1)^1\times 2^{-126}\times 0.111\cdots 11=-2^{-126}\times(1-2^{-23})$	$(-1)^1\times 2^{-1022}\times 0.111\cdots 11=-2^{-1022}\times(1-2^{-52})$

IEEE 754 浮点数表示范围

- 十进制小数转换成二进制小数时，可能出现**无限循环小数**，只能转换到符合精度要求的位数（即所谓的精度缺失）。
- IEEE 754浮点格式位数有限（单精度32位、双精度64位），无法精确表示无限循环小数。
  - 通常只能采用舍入的方式近似表示，因此会带来数据表示的误差。这种误差会在计算的过程中不断累积放大，可能导致严重后果。

【举例】采用双精度浮点数运算

$$0.1 + 0.1 + 0.1 - 0.3 \neq 0$$

$$0.7 - 0.2 \neq 0.5$$

$$3.3 \div 1.1 \neq 3$$

- 综上所述，程序员使用二进制浮点数编程时一定要非常小心，要充分考虑浮点数运算可能带来的计算误差，尽量避免对浮点数进行直接比较，在一些对误差极其敏感的情况下，建议采用十进制浮点数进行运算。

IEEE 754 数据格式规范

标准	C语言数据类型	位宽	基数	阶码位数	阶码偏移常数	尾数位数	最大值量级
半精度	_Float16	16	2	5	15	10	10^5
单精度	float	32	2	8	127	23	10^{38}
扩展单精度		≥ 43	2	≥ 11		≥ 31	
双精度	double	64	2	11	1023	52	10^{308}
扩展双精度	浮点运算器内部	≥ 79	2	≥ 15		≥ 63	
四精度	long double	128	2	15	16383	112	10^{4932}
八精度	暂未支持	256	2	19	262143	236	10^{78913}
Decimal32	_Decimal32	32	10	6	101	20	10^{96}
Decimal64	_Decimal64	64	10	8	398	50	10^{384}
Decimal128	_Decimal128	128	10	12	6176	110	10^{6144}

注：带有背景色的浮点数格式为 IEEE 754 – 2008 新增标准。

IEEE 754 单精度（32位）浮点数与对应真值之间的转换

【例题1】将十进制数408.6875转换成IEEE 754单精度浮点数的十六进制机器码。

解析：

408.6875 —转二进制→ 110011000.1011 —尾数变成$1.M$形式→ $2^8 \times 1.100110001011$

符号S = 0 表示正数，阶码E = 8 + 127 = 135 = 10000111，尾数M = 100110001011

0	10000111	10011000101100000000000
符号S	阶码E	尾数M
(1位)	(8位)	(23位)

0100 0011 1100 1100 0101 1000 0000 0000

4 3 C C 5 8 0 0

IEEE 754 单精度（32位）浮点数与对应真值之间的转换

【例题2】若$C1830000$是某个IEEE 754单精度浮点数的十六进制机器码，求其对应的十进制值。

解析：

$C1830000$ —转二进制→ $1100\ 0001\ 1000\ 0011\ 0000\ 0000\ 0000\ 0000$

符号$S = 1$，

阶码$E = 100\ 0001\ 1$，

阶码的真值x = 阶码$E - 127$

$= 131 - 127$

$= 4$

尾数$M = 000\ 0011\ 0000\ 0000\ 0000\ 0000$

尾数的$1.M$形式 $= 1.0000011$

浮点数的真值 $= (-1)^S \times 2^x \times 1.M = (-1)^1 \times 2^4 \times 1.0000011$

$= -10000.011$

$= -16.375$

IEEE 754 单精度（32位）浮点数与对应真值之间的转换

【2011年 题13】float型数据通常用IEEE 754单精度格式表示。若编译器将float型变量x分配一个32位浮点寄存器FR1中，且$x=-8.25$，则FR1的内容是（A）。

A. $C104\ 0000H$　　B. $C242\ 0000H$　　C. $C184\ 0000H$　　D. $C1C2\ 0000H$

解析：

-8.25 —转二进制→ -1000.01 —尾数变成$1.M$形式→ $2^3\times(-1.00001)$

符号$S=1$ 表示负数，阶码$E=3+127=130=10000010$，尾数$M=00001$

1	10000010	00001 00000000000000000
符号S	阶码E	尾数M
(1位)	(8位)	(23位)

1100 0001 0000 0100 0000 0000 0000 0000

C 1 0 4 0 0 0 0

IEEE 754 单精度（32位）浮点数与对应真值之间的转换

【2013年 题13】某数采用IEEE 754单精度浮点数格式表示为$C640\ 0000H$，则该数的值是（A）。

A. -1.5×2^{13}　　B. -1.5×2^{12}　　C. -0.5×2^{13}　　D. -0.5×2^{12}

解析：

$C640\ 0000$ —转二进制→ 1100 0110 0100 0000 0000 0000 0000 0000

符号$S=1$；

阶码$E=100\ 0110\ 0$

阶码的真值$x=$阶码$E-127$

$=140-127$

$=13$

尾数$M=100\ 0000\ 0000\ 0000\ 0000\ 0000$

尾数的$1.M$形式$=1.1$

浮点数的真值 $=(-1)^S\times2^x\times1.M=(-1)^1\times2^{13}\times1.1$

$=-1.5\times2^{13}$

IEEE 754 单精度（32位）浮点数与对应真值之间的转换

【2014年 题14】float型数据常用IEEE 754单精度浮点格式表示。假设两个float型变量x和y分别存放在32位寄存器$f1$和$f2$中，若$(f1)=CC90\ 0000H$，$(f2)=B0C0\ 0000H$，则x和y之间的关系是（A）。

A. $x<y$且符号相同　　B. $x<y$且符号不同

C. $x>y$且符号相同　　D. $x>y$且符号不同

解析：

$CC900000$ —转二进制→ $1100\ 1100\ 1001\ 0000\ 0000\ 0000\ 0000\ 0000$

符号$S=1$，

阶码$E=100\ 1100\ 1$，

阶码的真值x = 阶码$E-127$

$=153-127$

$=26$

尾数$M=001\ 0000\ 0000\ 0000\ 0000\ 0000$

尾数的$1.M$形式 $=1.001$

浮点数x的真值 $=(-1)^S\times 2^x\times 1.M=(-1)^1\times 2^{26}\times 1.001=-1.125\times 2^{26}$

IEEE 754 单精度（32位）浮点数与对应真值之间的转换

【2014年 题14】float型数据常用IEEE 754单精度浮点格式表示。假设两个float型变量x和y分别存放在32位寄存器$f1$和$f2$中，若$(f1)=CC90\ 0000H$，$(f2)=B0C0\ 0000H$，则x和y之间的关系是（A）。

A. $x<y$且符号相同　　B. $x<y$且符号不同

C. $x>y$且符号相同　　D. $x>y$且符号不同

解析：

$B0C00000$ —转二进制→ $1011\ 0000\ 1100\ 0000\ 0000\ 0000\ 0000\ 0000$

符号$S=1$，

阶码$E=011\ 0000\ 1$，

阶码的真值$x=$阶码$E-127$

$=97-127$

$=-30$

尾数$M=100\ 0000\ 0000\ 0000\ 0000\ 0000$

尾数的$1.M$形式$=1.1$

浮点数y的真值 $=(-1)^S\times 2^x\times 1.M=(-1)^1\times 2^{-30}\times 1.1=-1.5\times 2^{-30}$

IEEE 754 单精度（32位）浮点数与对应真值之间的转换

【2014年 题14】float型数据常用IEEE 754单精度浮点格式表示。假设两个float型变量x和y分别存放在32位寄存器$f1$和$f2$中，若$(f1)=CC90\ 0000H$，$(f2)=B0C0\ 0000H$，则x和y之间的关系是（A）。

A. $x<y$且符号相同

B. $x<y$且符号不同

C. $x>y$且符号相同

D. $x>y$且符号不同

解析：

浮点数x的真值 $=-1.125\times2^{26}$ $<$ 浮点数y的真值 $=-1.5\times2^{-30}$

IEEE 754 单精度（32位）浮点数与对应真值之间的转换

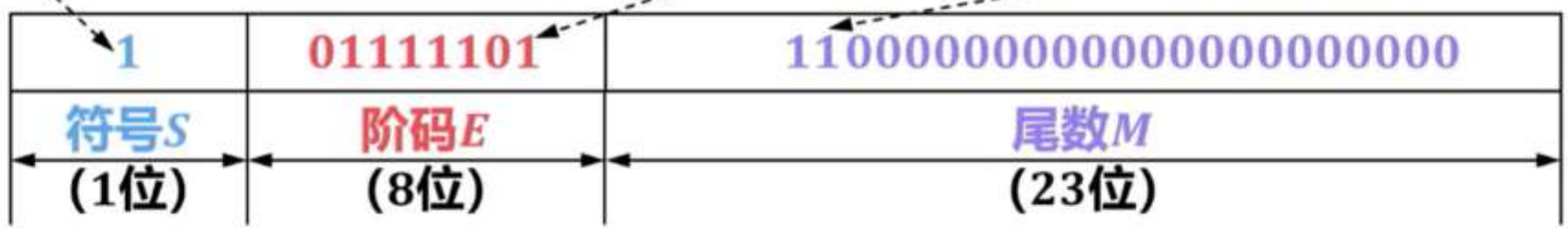
【2022年 题14】-0.4375的IEEE 754单精度浮点数表示为（A）。

A. $BEE0\ 0000H$　　B. $BF60\ 0000H$　　C. $BF70\ 0000H$　　D. $C0E0\ 0000H$

解析：

-0.4375 —转二进制→ -0.0111 —尾数变成$1.M$形式→ $2^{-2}\times(-1.11)$

符号$S=1$ 表示负数，阶码$E=(-2)+127=125=01111101$，尾数$M=11$



1011 1110 1110 0000 0000 0000 0000 0000

B E E 0 0 0 0 0

2-5 C语言中的数据类型及转换



计算机中的数据类型

- 计算机中的数据以二进制的形式存储在寄存器或存储器中。
- 汇编语言中的数据类型取决于**指令操作码**。
  - 存储在寄存器、存储器中的**操作数本身没有数据类型**，对该数进行何种数据类型的操作完全**取决于指令**。
  - 同一个操作数，既可以当作有符号数，也可以当作无符号数；既可以是定点数，也可以是浮点数。

【举例】

32位寄存器

b_{31}	b_{30}	b_{29}	…	b_1	b_0
0	0	0	…	0	0
0	0	0	…	0	1
0	0	0	…	1	1
⋮					
1	1	1	…	1	1

2^{32} 个组合

- 机器怎么知道这些数据是定点数还是浮点数?
- 如果是定点数，是有符号数还是无符号数?



计算机中的数据类型

- 计算机中的数据以二进制的形式存储在寄存器或存储器中。
- 汇编语言中的数据类型取决于**指令操作码**。
  - 存储在寄存器、存储器中的**操作数本身没有数据类型**，对该数进行何种数据类型的操作完全**取决于指令**。
  - 同一个操作数，既可以当作有符号数，也可以当作无符号数；既可以是定点数，也可以是浮点数。
- 高级语言具有数据类型，下面以C语言为例进行介绍。

C语言中常见的整型数据类型

char	8 位
short	16 位
int	32 位
long	64 位

默认表示有符号整数

前面加上“unsigned”表示无符号数

C语言中主要的浮点数据类型

float	对应IEEE 754单精度（32位）浮点数标准
double	对应IEEE 754双精度（64位）浮点数标准

C语言中整型变量的取值范围

		char（8位）	short（16位）	int（32位）	long（64位）
无符号(unsigned)最大值	机器码	0xFF	0xFFFF	0xFFFF FFFF	0xFFFF FFFF FFFF FFFF
	真值	255	65,535	4,294,967,295	18,446,744,073,709,551,615
有符号最大值	机器码	0x7F	0x7FFF	0x7FFF FFFF	0x7FFF FFFF FFFF FFFF
	真值	127	32,767	2,147,483,647	9,223,372,036,854,775,807
有符号最小值	机器码	0x80	0x8000	0x8000 0000	0x8000 0000 0000 0000
	真值	−128	−32,768	−2,147,483,648	−9,223,372,036,854,775,808
−1	机器码	0xFF	0xFFFF	0xFFFF FFFF	0xFFFF FFFF FFFF FFFF
0	机器码	0x00	0x0000	0x0000 0000	0x0000 0000 0000 0000

■ 无论是无符号数还是有符号数，C语言程序并不检测数据在加、减、乘等运算中产生的溢出现象。

□ 程序员应尽量避免出现这种情况，所编制的应用程序应具有对溢出进行判断的功能。

C语言中数据类型转换

■ char、short、int、long这4种整型数据的表示范围不一样，很可能数据转换后精度缺失，此时就只能尽量保持转换前后的机器码相同或机器码部分相同。

相同字长之间	两种整型数据交集部分相互转换	无差错
	两种整型数据交集以外相互转换	有差错
小字长转大字长	原数据为无符号数，进行0扩展	无差错
	原数据为有符号数，进行符号扩展	
大字长转小字长	截短处理，表示范围缩小	很可能出错

■ int、float、double之间也可以进行强制类型转换。

- ☐ 上述3种类型数据的机器码并不相同（int型数据是32位有符号整数，用补码表示；float和double型数据分别是32位和64位浮点数，它们的阶码用移码表示、尾数用原码表示）。
- ☐ 上述3种类型数据的表示范围和精度也不相同。
- ☐ 因此在转换过程中编译器只能保证数值尽量相等，大多数情况下只是近似值。

float → double	值相同
double → float	大数可能溢出，高精度发生舍入
float/double → int	小数部分向0截断，大数可能溢出
int → float	比较大的数无法精确表示
int → double	值相同

3-1-1 移位运算 —— 逻辑移位



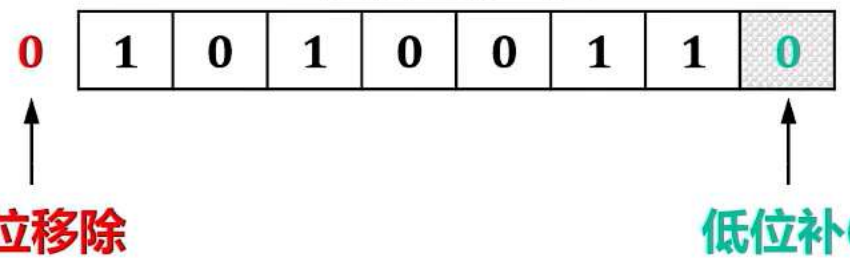
逻辑移位

- 逻辑移位的对象：无符号数
- 逻辑移位的规则：
  - 逻辑左移：高位移除，低位补0；
  - 逻辑右移：低位移除，高位补0；

【举例】

	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
寄存器	0	1	0	1	0	0	1	1

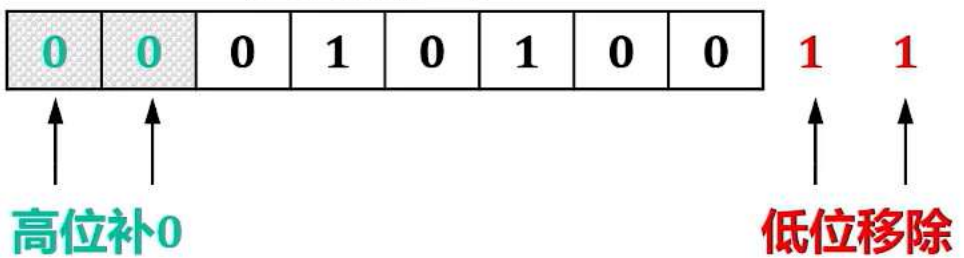
逻辑左移1位



【举例】

	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
寄存器	0	1	0	1	0	0	1	1

逻辑右移2位



3-1-2 移位运算 —— 算术移位



算术移位

真　值	机 器 码	空位添补	丢 位 情 况	算术移位效果
正　数	原码=补码=反码	0	最高位丢1，则结果出错 最低位丢1，则精度缺失	在未出现因丢位产生结果出错或精度缺失的情况下： ● 左移：乘以2^n ● 右移：除以2^n 其中，n为移位次数。
负　数	原码			
	反码	1	最高位丢0，则结果出错 最低位丢0，则精度缺失	
	补码	左移补0	最高位丢0，则结果出错	
		右移补1	最低位丢1，则精度缺失	

* 补码的负数算术右移补符号位

算术移位

- 算术移位的对象：有符号数 （针对定点数，包括定点整数和定点小数）
- 不论正数还是负数，符号位保持不变，仅对数值位进行移位（左移或右移）。
- 对真值的原码、反码和补码进行算术移位后，它们各自所对应的新的真值应该保持一致。
  - ☐ 当真值为正数时，真值的原码、反码和补码都相同，因此，对它们进行算术移位后，它们各自所对应的新的真值自然是保持一致的。对于移位后出现的空位，规定添补0。
  - ☐ 当真值为负数时，真值的原码、反码和补码都不同，因此，对它们进行算术移位后，为了确保它们各自所对应的新的真值保持一致，对于移位后出现的空位，添补规则各不相同。

真 值	机 器 码	空位添补	丢 位 情 况	算术移位效果
正 数	原码=补码=反码	0	最高位丢1，则结果出错 最低位丢1，则精度缺失	在未出现因丢位产生结果出错或精度缺失的情况下： ● 左移：乘以2^n ● 右移：除以2^n 其中，n为移位次数。
负 数	原码			
	反码	1	最高位丢0，则结果出错 最低位丢0，则精度缺失	
	补码	左移补0	最高位丢0，则结果出错	
		右移补1	最低位丢1，则精度缺失	

算术移位

- 有符号定点数的补码的另一种算术移位方法，即“符号位也参与移位”，具体规则如下：
  - 左移：高位移除，低位添补0；移动前后若符号位发生变化，则发生溢出。
  - 右移：低位移除，高位添补符号。
- 上述这种针对有符号定点数的补码的算术移位方法，具有以下优点：
  - 左移时，有检测出发生溢出的方法：符号位发生变化可判定溢出。
  - 符号位与数值位一起移位，方便ALU处理，也方便人们记忆。

算术移位的应用举例

假设 $x=-26$，仅用加法和算术移位操作计算 $x\times 3$ 。

	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0	
$[x]_{补}$	1	1	1	0	0	1	1	0	$x=-26$
对 $[x]_{补}$ 算术左移1位	1	1	0	0	1	1	0	0	$a=-52$

乘以2^1

加法

1	0	1	1	0	0	1	0	-78

C语言中的移位运算

- 至此，我们已经介绍了针对无符号数的逻辑移位（上节课）、针对有符号定点数（包括定点整数和定点小数）的算术移位（本节课）。
- C语言中的移位运算操作符包括“<<”和“>>”两种，分别表示左移和右移。
  - 左移运算操作符“<<”对应汇编指令中的逻辑左移，即高位移除，低位补0。
  - 右移运算操作符“>>”则根据操作数是无符号还是有符号类型分别对应汇编指令中的逻辑右移和算术右移指令。
    - 逻辑右移：将低位移除，高位补0。
    - 算术右移：将低位移除，高位补原数据的符号位。

```
    int si = -1;
00101795  mov         dword ptr [si],0FFFFFFFFh
    unsigned int usi = 1;
0010179C  mov         dword ptr [usi],1
    si = si << 6;
001017A3  mov         eax,dword ptr [si]
001017A6  shl         eax,6          逻辑左移
001017A9  mov         dword ptr [si],eax
    si = si >> 7;
001017AC  mov         eax,dword ptr [si]
001017AF  sar         eax,7          算术右移
001017B2  mov         dword ptr [si],eax
    usi = usi << 6;
001017B5  mov         eax,dword ptr [usi]
001017B8  shl         eax,6          逻辑左移
001017BB  mov         dword ptr [usi],eax
    usi = usi >> 7;
001017BE  mov         eax,dword ptr [usi]
001017C1  shr         eax,7          逻辑右移
001017C4  mov         dword ptr [usi],eax
```

3-1-3 移位运算 —— 循环移位



循环移位

- 循环移位的对象：**无符号数**
  - 将无符号数二进制形式中的各个位向左或向右移动，被移出的位会重新出现在另一端，形成循环。
- 在很多处理器架构中，循环移位指令会影响状态寄存器中的进位标志CF（Carry Flag）位，CF标志位用于标识在执行算术或逻辑操作时是否发生了进位。
- 根据CF标志位是否加入循环移位过程，循环移位可分为以下四种：
  - 不带CF标志位的循环右移
  - 不带CF标志位的循环左移

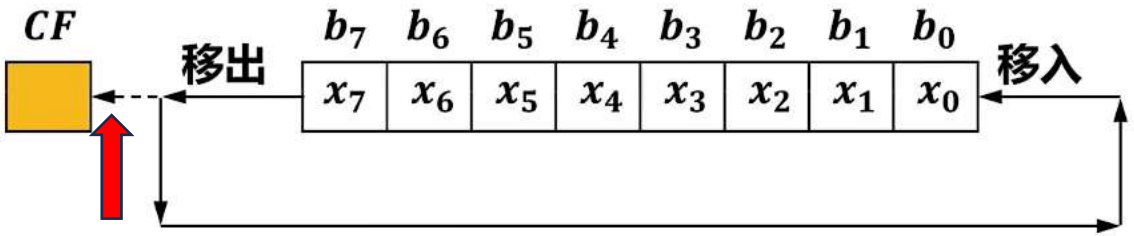
  } 小循环

  - 带CF标志位的循环右移
  - 带CF标志位的循环左移

  } 大循环

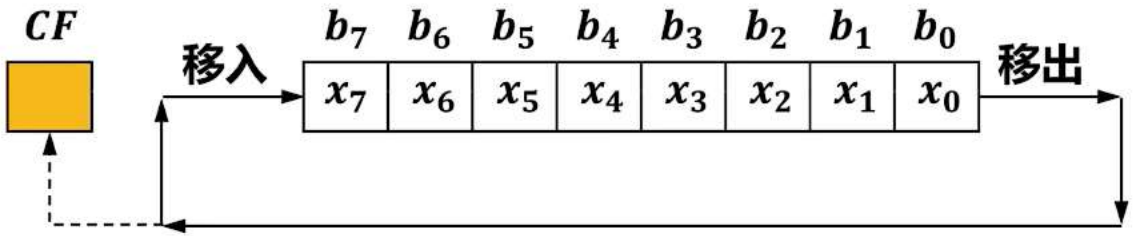
循环移位

不带CF标志位的循环左移

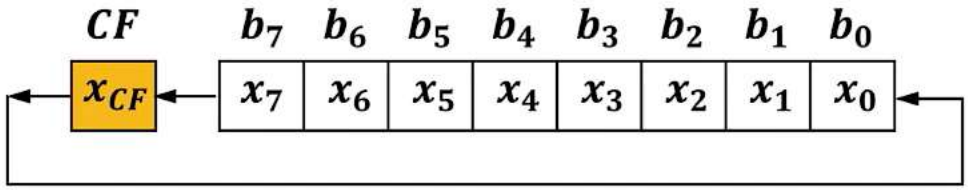


移出的数据也会放入标记位

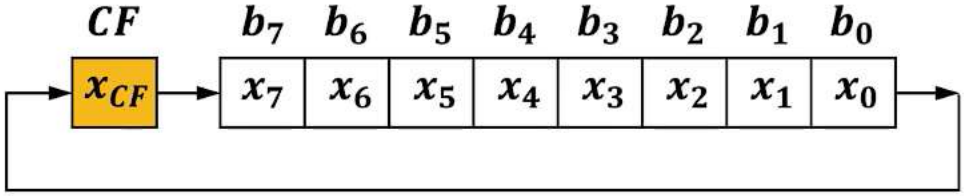
不带CF标志位的循环右移



带CF标志位的循环左移



带CF标志位的循环右移



循环移位的应用

- 循环移位的应用主要有：**加密算法**、**哈希函数**、**优化算法**

  - 加密算法：通过循环移位可以实现数据的混淆和置换，增强加密算法的安全性。
  - 哈希函数：通过循环移位可以用来改变输入数据的排列顺序，以产生不同的哈希值，有利于增哈希函数的混淆性和扩散性。
  - 优化算法：在某些算法中，循环移位可以用于优化性能和节省资源。例如，在图形处理和数字信号处理中，循环移位可以用于加速算法的执行。

3-2-1 定点数的加法和减法运算
—— 补码加减法运算公式



补码加减法运算公式

■ **补码加法和减法运算的公式如下：**

$$[A]_{补} + [B]_{补} = [A + B]_{补}$$

$$[A]_{补} - [B]_{补} = [A - B]_{补} = [A]_{补} + [-B]_{补}$$

$(mod\ M)$ 的模数 M：

- 定点整数：$M = 2^{n+1}$
- 定点小数：$M = 2^1 = 2$

其中，n为补码数值位的位数

【例题1】 设$x = 0.0101$，$y = 0.0001$，求 $[x]_{补} + [y]_{补}$。

解析：

$$\begin{array}{ll} [x]_{补} & 0.0101 \\ [y]_{补} & 0.0001 \\ + & \\ \hline & 0.0110 \end{array}$$

相同

$$[x+y]_{补} = [0.0101 + 0.0001]_{补}$$

$$= [0.0110]_{补} = 0.0110$$

【例题2】 设$x = -0.0101$，$y = -0.0001$，求 $[x]_{补} + [y]_{补}$。

解析：

$$\begin{array}{ll} [x]_{补} & 1.1011 \\ [y]_{补} & 1.1111 \\ + & \\ \hline & 11.1010 \end{array}$$

符号位进位的值为模数，应舍去

相同

$$[x+y]_{补} = [(-0.0101) + (-0.0001)]_{补}$$

$$= [-0.0110]_{补} = 1.1010$$

补码加减法运算公式

■ **补码加法和减法运算的公式如下：**

$$[A]_{补} + [B]_{补} = [A + B]_{补}$$

$$[A]_{补} - [B]_{补} = [A - B]_{补} = [A]_{补} + [-B]_{补}$$

$(mod\ M)$ 的模数 M：

- 定点整数：$M = 2^{n+1}$
- 定点小数：$M = 2^1 = 2$

其中，n为补码数值位的位数

【例题2】设$x = -0.0101$，$y = -0.0001$，求 $[x]_{补} + [y]_{补}$。

解析：

$[x]_{补}$　1．1 0 1 1

$[y]_{补}$　1．1 1 1 1

\+

符号位进位的值为模数，应舍去 → 1 1．1 0 1 0 ← 相同

$$[x+y]_{补} = [(-0.0101) + (-0.0001)]_{补}$$

$$= [-0.0110]_{补} = 1.1010$$

【例题3】设$x = 0.1011$，$y = -0.0101$，求 $[x]_{补} + [y]_{补}$。

解析：

$[x]_{补}$　0．1 0 1 1

$[y]_{补}$　1．1 0 1 1

\+

符号位进位的值为模数，应舍去 → 1 0．0 1 1 0 ← 相同

$$[x+y]_{补} = [0.1011 + (-0.0101)]_{补}$$

$$= [0.0110]_{补} = 0.0110$$

补码加减法运算公式

■ **补码加法和减法运算的公式如下：**

$$[A]_{补} + [B]_{补} = [A + B]_{补}$$

$$[A]_{补} - [B]_{补} = [A - B]_{补} = [A]_{补} + [-B]_{补}$$

$(mod\ M)$ 的模数 M：

- 定点整数：$M = 2^{n+1}$
- 定点小数：$M = 2^1 = 2$

其中，n为补码数值位的位数

【例题4】设$x = 0.1001$，$y = 0.0110$，求 $[x]_{补} - [y]_{补}$。

解析：

$$[x]_{补} \quad 0.1001$$
$$[-y]_{补} \quad 1.1010$$
$$+$$

符号位进位的值为模数，应舍去 → $1\ 0.0011$ ← 相同

$$[x-y]_{补} = [0.1001 - 0.0110]_{补}$$
$$= [0.0011]_{补} = 0.0011$$

【例题5】设$x = -0.0110$，$y = -0.1001$，求 $[x]_{补} - [y]_{补}$。

解析：

$$[x]_{补} \quad 1.1010$$
$$[-y]_{补} \quad 0.1001$$
$$+$$

符号位进位的值为模数，应舍去 → $1\ 0.0011$ ← 相同

$$[x-y]_{补} = [-0.0110 - (-0.1001)]_{补}$$
$$= [0.0011]_{补} = 0.0011$$

补码加减法运算公式

■ **补码加法和减法运算的公式如下：**

$$[A]_{补} + [B]_{补} = [A + B]_{补}$$

$$[A]_{补} - [B]_{补} = [A - B]_{补} = [A]_{补} + [-B]_{补}$$

$(mod\ M)$ 的模 M：

- 定点整数：$M = 2^{n+1}$
- 定点小数：$M = 2^1 = 2$

其中，n为补码数值位的位数

【习题1】 设$x = +1001$，$y = +0101$，求 $[x]_{补} + [y]_{补}$。

解析：

$$\begin{array}{ll} [x]_{补} & 0,\ 1\ 0\ 0\ 1 \\ [y]_{补} & 0,\ 0\ 1\ 0\ 1 \\ + & \\ \hline & 0,\ 1\ 1\ 1\ 0 \end{array}$$

$$[x+y]_{补} = [1001 + 0101]_{补}$$

$$= [1110]_{补} = 0,1110$$

【习题2】 设$x = -1001$，$y = -0101$，求 $[x]_{补} + [y]_{补}$。

解析：

$$\begin{array}{ll} [x]_{补} & 1,\ 0\ 1\ 1\ 1 \\ [y]_{补} & 1,\ 1\ 0\ 1\ 1 \\ + & \\ \hline & 1\ 1,\ 0\ 0\ 1\ 0 \end{array}$$

符号位进位的值为模数，应舍去

$$[x+y]_{补} = [(-1001) + (-0101)]_{补}$$

$$= [-1110]_{补} = 1,0010$$

补码加减法运算公式

■ **补码加法和减法运算的公式如下：**

$$[A]_{补} + [B]_{补} = [A + B]_{补}$$

$$[A]_{补} - [B]_{补} = [A - B]_{补} = [A]_{补} + [-B]_{补}$$

$(mod\ M)$ 定点整数：$M = 2^{n+1}$

$(mod\ M)$ 定点小数：$M = 2^1 = 2$

其中，n为补码数值位的位数

【习题3】设$x = +1001$，$y = -0101$，
求$[x]_{补} + [y]_{补}$。

解析：

$[x]_{补}$　　0，1 0 0 1

$[y]_{补}$　　1，1 0 1 1

\+

符号位进位的值为模数，应舍去 → 1　0，0 1 0 0　← 相同

$[x + y]_{补} = [1001 + (-0101)]_{补}$

$= [0100]_{补} = 0,0100$

补码加减法运算公式

■ **补码加法和减法运算的公式如下：**

$$[A]_{补} + [B]_{补} = [A + B]_{补}$$

$$[A]_{补} - [B]_{补} = [A - B]_{补} = [A]_{补} + [-B]_{补}$$

$(mod\ M)$ 定点整数：$M = 2^{n+1}$

定点小数：$M = 2^1 = 2$

其中，n为补码数值位的位数

【习题4】 设$x = +1001$，$y = +0110$，求$[x]_{补} - [y]_{补}$。

解析：

$[x]_{补}$　0，1 0 0 1

$[-y]_{补}$　1，1 0 1 0

\+

符号位进位的值为模数，应舍去 → 1　0，0 0 1 1 ← 相同

$[x - y]_{补} = [1001 - 0110]_{补}$

$= [0011]_{补} = 0,0011$

【习题5】 设$x = -1001$，$y = -0110$，求$[x]_{补} - [y]_{补}$。

解析：

$[x]_{补}$　1，0 1 1 1

$[-y]_{补}$　0，0 1 1 0

\+

1，1 1 0 1 ← 相同

$[x - y]_{补} = [-1001 - (-0110)]_{补}$

$= [-0011]_{补} = 1,1101$

补码加减法运算公式

■ 经过第2章的学习可知，定点数（定点整数和定点小数）在计算机内部采用补码表示，原因如下：

- □ 补码的符号位可以与数值位一起参加运算。
- □ 采用补码可将减法运算转换成加法运算。

} 运算规则简单，易于实现

■ 补码加法和减法运算的公式如下：

$$[A]_{补} + [B]_{补} = [A+B]_{补}$$

$$[A]_{补} - [B]_{补} = [A-B]_{补} = [A]_{补} + [-B]_{补}$$

$$(mod\ M)\begin{cases} \text{定点整数：} M = 2^{n+1} \\ \text{定点小数：} M = 2^1 = 2 \end{cases}$$

其中，n为补码数值位的位数

可利用下面的定点数补码的定义式证明

定点整数的补码定义

假设真值x为定点整数，n为x的补码表示中数值位的位数（比特数量），加上1个符号位，x的补码表示共有$n+1$位，最低位的位权值为2^0，而最高位（符号位）的位权值为2^n，因此最高位进位的位权值为2^{n+1}，即模数为2^{n+1}。

$$[x]_{补} = \begin{cases} 0, x & 0 \le x < 2^n \\ 2^{n+1} + x & -2^n \le x < 0 \quad (mod\ 2^{n+1}) \end{cases}$$

定点小数的补码定义

假设真值x为定点小数（纯小数），小数点左侧的位为最高位（符号位），其位权值为2^0，而最高位进位的位权值为2^1，即模数为$2^1 = 2$。

$$[x]_{补} = \begin{cases} x & 0 \le x < 1 \\ 2 + x & -1 \le x < 0 \quad (mod\ 2) \end{cases}$$

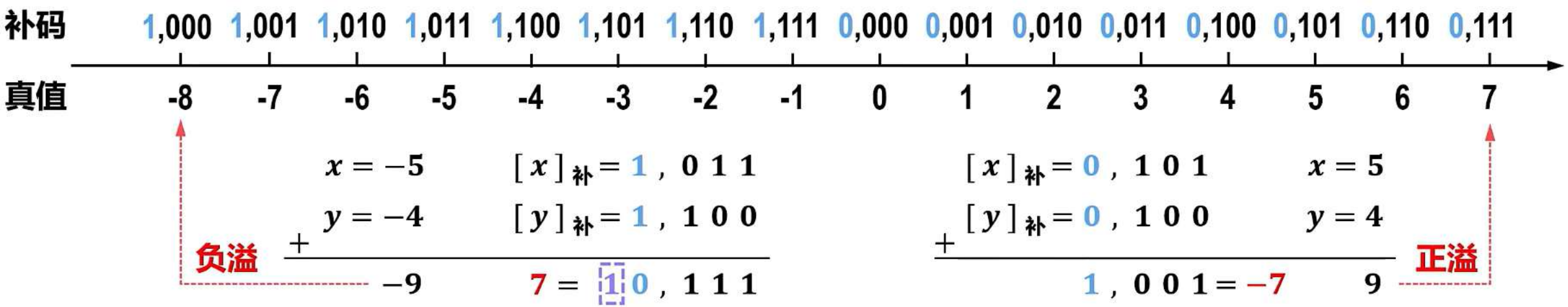
3-2-2 定点数的加法和减法运算
—— 补码加减法运算的溢出检测



溢出的概念

- 计算机的字长是有限的，因此所能表示的数据范围也是有限的。
- 当运算结果超出所能表示的数据范围时，就会出现溢出（Overflow）。
  - 溢出会导致错误的运算结果。
  - 计算机系统设计人员必须要解决溢出的检测问题，以便在发生溢出时计算机能做出相应的处理。

【举例】 计算机的字长为4位



溢出检测

- 定点数补码加减运算判断溢出的方法主要有以下3种：

方法1 根据操作数的符号位与运算结果的符号位是否一致进行判断

方法2 根据运算过程中最高数值位的进位与符号位的进位是否一致进行判断

方法3 利用变形补码（具有2位符号位的补码）的符号位进行判断

溢出检测

方法1 根据**操作数的符号位**与**运算结果的符号位**是否一致进行判断

两个操作数**相加**时，当它们的**符号位相同**（即同正或同负）时，才**可能**发生**溢出**。

- **溢出**：运算结果的符号位与原操作数的**符号位不同**
- **未溢出**：运算结果的符号位与原操作数的**符号位相同**

【例题1】若 $x=-101$，$y=-011$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。

$$
\begin{array}{rl}
[x]_{补} = & \boxed{1},\ 0\ 1\ 1 \\
[y]_{补} = & \boxed{1},\ 1\ 0\ 1 \\
+ & \\
\hline
 & \boxed{1}\ \boxed{1},\ 0\ 0\ 0
\end{array}
$$

符号位相同，未溢出

符号位进位的值为模数，应舍弃

【例题2】若 $x=+101$，$y=+011$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。

$$
\begin{array}{rl}
[x]_{补} = & \boxed{0},\ 1\ 0\ 1 \\
[y]_{补} = & \boxed{0},\ 0\ 1\ 1 \\
+ & \\
\hline
 & \boxed{1},\ 0\ 0\ 0
\end{array}
$$

符号位不同，溢出

溢出检测

方法1 根据**操作数的符号位**与**运算结果的符号位**是否一致进行判断

两个操作数**相加**时，当它们的**符号位相同**（即同正或同负）时，才**可能**发生**溢出**。

- **溢出**：运算结果的符号位与原操作数的**符号位不同**
- **未溢出**：运算结果的符号位与原操作数的**符号位相同**

【练习1】若 $x=-111$，$y=-010$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。

$$
\begin{array}{rl}
[x]_{补} = & \boxed{1},\ 0\ 0\ 1 \\
[y]_{补} = & \boxed{1},\ 1\ 1\ 0 \\
+ & \\
\hline
 & \boxed{1}\ \boxed{0},\ 1\ 1\ 1
\end{array}
$$

符号位不同，溢出

符号位进位的值为模数，应舍弃

【练习2】若 $x=+011$，$y=+100$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。

$$
\begin{array}{rl}
[x]_{补} = & \boxed{0},\ 0\ 1\ 1 \\
[y]_{补} = & \boxed{0},\ 1\ 0\ 0 \\
+ & \\
\hline
 & \boxed{0},\ 1\ 1\ 1
\end{array}
$$

符号位相同，未溢出

溢出检测

方法1 根据**操作数的符号位**与**运算结果的符号位**是否一致进行判断

两个操作数**相加**时，当它们的**符号位相同**（即同正或同负）时，才**可能**发生**溢出**。

- **溢出**：运算结果的符号位与原操作数的**符号位不同**
- **未溢出**：运算结果的符号位与原操作数的**符号位相同**

由于采用补码可将减法运算转换成加法运算，因此不论作加法还是减法，只要实际参加运算的两个操作数符号相同，但运算结果的符号位与原操数不同，即为溢出。

【举例】若$x=+101$，$y=-011$，采用4位补码表示，判断$[x-y]_{补}$是否会溢出。

$$
\begin{array}{rl}
[x]_{补} & = 0,\ 1\ 0\ 1 \\
[-y]_{补} & = 0,\ 0\ 1\ 1 \\
+ & \\
\hline
 & 1,\ 0\ 0\ 0
\end{array}
$$

符号位不同，溢出

本例中，$-y$才是实际参加运算的操作数

溢出检测

方法1 根据操作数的符号位与运算结果的符号位是否一致进行判断

两个操作数相加时，当它们的符号位相同（即同正或同负）时，才可能发生溢出。

- 溢出：运算结果的符号位与原操作数的符号位不同
- 未溢出：运算结果的符号位与原操作数的符号位相同

由于采用补码可将减法运算转换成加法运算，因此不论作加法还是减法，只要实际参加运算的两个操作数符号相同，但运算结果的符号位与原操数不同，即为溢出。

操作数 x 的符号位记为 x_{n-1}，操作数 y 的符号位记为 y_{n-1}，运算结果 s 的符号位记为 s_{n-1}，

溢出标志位记为 OF，当 OF 为1时表示发生溢出。

x 与 y 的符号位相同，与 s 的符号位相反

$$x_{n-1} \cdot y_{n-1} \cdot \overline{s}_{n-1}$$

$$\overline{x}_{n-1} \cdot \overline{y}_{n-1} \cdot s_{n-1}$$

$$OF = x_{n-1} \cdot y_{n-1} \cdot \overline{s}_{n-1} + \overline{x}_{n-1} \cdot \overline{y}_{n-1} \cdot s_{n-1}$$

$$OF = 1 \cdot 1 \cdot \overline{1} + \overline{1} \cdot \overline{1} \cdot 1 = 0$$

【例题1】若 $x = -101$，$y = -011$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。

$$[x]_{补} = \overset{x_{n-1}}{1},\ 0\ 1\ 1$$

$$[y]_{补} = \overset{y_{n-1}}{1},\ 1\ 0\ 1$$

$$+$$

$$\overset{1}{1},\ \overset{s_{n-1}}{0}\ 0\ 0$$

符号位相同，未溢出

符号位进位的值为模数，应舍弃

溢出检测

方法1 根据**操作数的符号位**与**运算结果的符号位**是否一致进行判断

两个操作数**相加**时，当它们的**符号位相同**（即同正或同负）时，才**可能**发生**溢出**。

- **溢出**：运算结果的符号位与原操作数的**符号位不同**
- **未溢出**：运算结果的符号位与原操作数的**符号位相同**

由于采用补码可将减法运算转换成加法运算，因此不论作加法还是减法，只要实际参加运算的两个操作数符号相同，但运算结果的符号位与原操数不同，即为溢出。

操作数 x 的符号位记为 x_{n-1}，操作数 y 的符号位记为 y_{n-1}，运算结果 s 的符号位记为 s_{n-1}，

溢出标志位记为 OF，当 OF **为1时表示发生溢出**。

x与y的符号位相同，与s的符号位相反

$$x_{n-1} \cdot y_{n-1} \cdot \overline{s}_{n-1}$$

$$\overline{x}_{n-1} \cdot \overline{y}_{n-1} \cdot s_{n-1}$$

$$OF = x_{n-1} \cdot y_{n-1} \cdot \overline{s}_{n-1} + \overline{x}_{n-1} \cdot \overline{y}_{n-1} \cdot s_{n-1}$$

$$OF = 0 \cdot 0 \cdot \overline{1} + \overline{0} \cdot \overline{0} \cdot 1 = 1$$

【例题2】若 $x = +101$，$y = +011$，采用4位补码表示，判断 $[x+y]_{补}$ 是否会溢出。

$$[x]_{补} = \overset{x_{n-1}}{0}, 101$$

$$[y]_{补} = \overset{y_{n-1}}{0}, 011$$

$$+$$

$$\overset{s_{n-1}}{1}, 000$$

符号位不同，溢出

溢出检测

方法2 根据运算过程中**最高数值位的进位**与**符号位的进位**是否一致进行判断

不同可判定为产生**溢出**，**相同**可判定为**没有产生溢出**。

【举例】 若$x=5$，$y=4$，采用4位补码表示，判断$[x+y]_{补}$是否会溢出。

符号位的进位 ← 0 ≠ 1 → 最高数值位的进位

进位

$x=5$ $[x]_{补}$ 0，1 0 1

$y=4$ $[y]_{补}$ 0，1 0 0

\+

9 正溢 1，0 0 1 = −7

【举例】 若$x=-5$，$y=-4$，采用4位补码表示，判断$[x+y]_{补}$是否会溢出。

符号位的进位 ← 1 ≠ 0 → 最高数值位的进位

进位

$x=-5$ $[x]_{补}$ 1，0 1 1

$y=-4$ $[y]_{补}$ 1，1 0 0

\+

负溢 −9 → [1] 0，1 1 1 = 7

符号位进位的值为模数，应舍弃

溢出检测

方法2 根据运算过程中**最高数值位的进位**与**符号位的进位**是否一致进行判断

不同可判定为产生**溢出**，**相同**可判定为**没有产生溢出**。

【举例】 若$x=5$，$y=-4$，采用4位补码表示，判断$[x+y]_{补}$是否会溢出。

符号位的进位 ← 1 = 1 → 最高数值位的进位

进位		1 = 1	
$x=5$	$[x]_{补}$	0，1 0 1	
$y=-4$	$[y]_{补}$	1，1 0 0	
+			
未溢出 1		1	0，0 0 1 = 1

符号位进位的值为模数，应舍弃

【举例】 若$x=5$，$y=-4$，采用4位补码表示，判断$[x-y]_{补}$是否会溢出。

符号位的进位 ← 0 ≠ 1 → 最高数值位的进位

进位		0 ≠ 1	
$x=5$	$[x]_{补}$	0，1 0 1	
$(-y)=4$	$[-y]_{补}$	0，1 0 0	
+			
9	正溢	1，0 0 1 = −7	

溢出检测

方法2 根据运算过程中**最高数值位的进位**与**符号位的进位**是否一致进行判断

不同可判定为产生**溢出**，**相同**可判定为**没有产生溢出**。

【练习3】若$x=-111$，$y=-010$，采用4位补码表示，判断$[x+y]_{补}$是否会溢出。

符号位的进位 ← 1≠0 → 最高数值位的进位

进位　1≠0　溢出

$[x]_{补}$　1，0 0 1

$[y]_{补}$　1，1 1 0

+

1　0，1 1 1

符号位进位的值为模数，应舍弃

【练习4】若$x=+100$，$y=+011$，采用4位补码表示，判断$[x-y]_{补}$是否会溢出。

符号位的进位 ← 1=1 → 最高数值位的进位

进位　1=1　未溢出

$[x]_{补}$　0，1 0 0

$[-y]_{补}$　1，1 0 1

+

1　0，0 0 1

符号位进位的值为模数，应舍弃

溢出检测

方法2 根据运算过程中**最高数值位的进位**与**符号位的进位**是否一致进行判断

不同可判定为产生**溢出**，**相同**可判定为**没有产生溢出**。

最高数值位的进位记为C_{n-1}， 符号位的进位记为C_n

溢出标志位记为OF，当OF为1时表示发生溢出。

C_{n-1}与C_n不同，则产生溢出（即OF为1）：

$$OF = C_{n-1} \oplus C_n$$

写出**溢出检测**的**逻辑表达式**?



溢出检测

方法3 利用**变形补码**（具有2位符号位的补码）的符号位进行判断

变形补码也称为**双符号补码**，具有2个符号位，其余与补码相同。

- 双符号位为 00 时，表示**正数**；
- 双符号位为 11 时，表示**负数**；
- 双符号位为 01 时，表示**正溢出**；
- 双符号位为 10 时，表示**负溢出**；

溢出判断非常直观，适合手工运算时的溢出检测，但其硬件成本高，在计算机中主要采用单符号溢出检测方案。

溢出检测

方法3 利用**变形补码**（具有2位符号位的补码）的符号位进行判断

- 双符号位为 00 时，表示**正数**；
- 双符号位为 01 时，表示**正溢出**；
- 双符号位为 11 时，表示**负数**；
- 双符号位为 10 时，表示**负溢出**；

【练习】(1) $[x]_{补}=00.1011$，$[y]_{补}=00.0111$，判断 $[x]_{补}+[y]_{补}$ 的结果是否溢出。

(2) $[x]_{补}=11.0101$，$[y]_{补}=11.0011$，判断 $[x]_{补}+[y]_{补}$ 的结果是否溢出。

(3) $[x]_{补}=00.1011$，$[y]_{补}=11.0011$，判断 $[x]_{补}+[y]_{补}$ 的结果是否溢出。

(1)

$$
\begin{array}{rl}
 & [x]_{补}\quad 00.1011 \\
 & [y]_{补}\quad 00.0111 \\
+ & \\
\hline
[x]_{补}+[y]_{补} & \boxed{01}.0010
\end{array}
$$

正溢出

(2)

$$
\begin{array}{rl}
 & [x]_{补}\quad 11.0101 \\
 & [y]_{补}\quad 11.0011 \\
+ & \\
\hline
[x]_{补}+[y]_{补} & \boxed{10}.1000
\end{array}
$$

负溢出

(3)

$$
\begin{array}{rl}
 & [x]_{补}\quad 00.1011 \\
 & [y]_{补}\quad 11.0011 \\
+ & \\
\hline
[x]_{补}+[y]_{补} & \boxed{11}.1110
\end{array}
$$

未溢出

溢出检测

方法3 利用**变形补码**（具有2位符号位的补码）的符号位进行判断

变形补码也称为**双符号补码**，具有2个符号位，其余与补码相同。

- 双符号位为 00 时，表示**正数**；
- 双符号位为 11 时，表示**负数**；
- 双符号位为 01 时，表示**正溢出**；
- 双符号位为 10 时，表示**负溢出**；

溢出判断非常直观，适合手工运算时的溢出检测，但硬件成本高，在机器内采用单符号溢出检测方案

注意：负数变形补码转换成原码仍可以使用反码法或扫描法。

写出**溢出检测**的**逻辑表达式**?



左起第一个符号位记为C_{n+1}

左起第二个符号位记为C_n

溢出标志位记为OF，当OF为1时表示发生溢出。

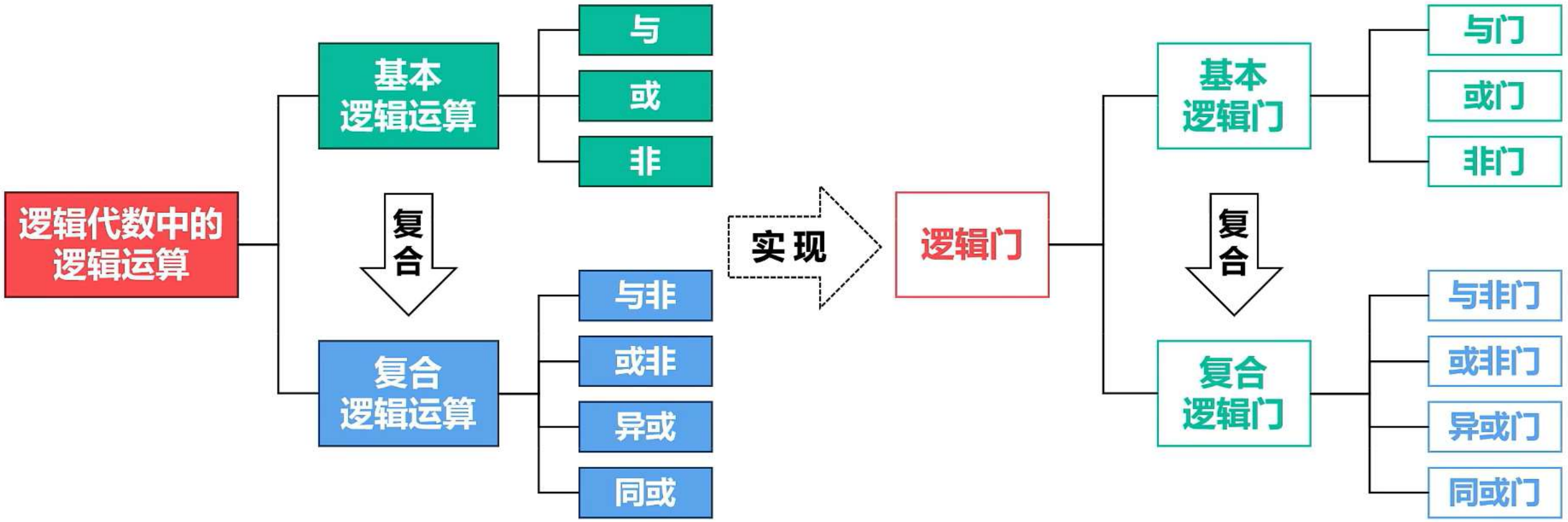
C_{n+1} 与 C_n 不同，则产生溢出（即OF为1）：

$$OF = C_{n+1} \oplus C_n$$

3-2-3 定点数的加法和减法运算
—— 逻辑代数和逻辑门



逻辑代数和逻辑门


逻辑代数中的逻辑运算
基本逻辑运算
与
或
非
复合
复合逻辑运算
与非
或非
异或
同或
实现
逻辑门
基本逻辑门
与门
或门
非门
复合
复合逻辑门
与非门
或非门
异或门
同或门

复合逻辑运算与复合逻辑门

与非运算 和 与非门 | 或非运算 和 或非门 | 异或运算 和 异或门 | 同或运算 和 同或门

与运算与非运算复合

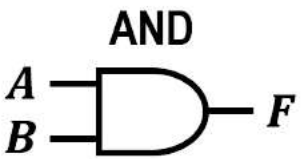
与运算

逻辑表达式：$F = A \cdot B = AB$

真值表：

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

电路符号：



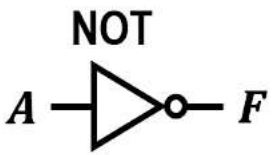
非运算

逻辑表达式：$F = \overline{A}$

真值表：

A	F
0	1
1	0

电路符号：



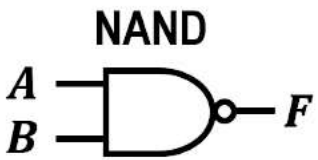
与非运算

逻辑表达式：$F = \overline{A \cdot B}$

真值表：

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0

电路符号：



复合逻辑运算与复合逻辑门

与非运算 和 与非门 | 或非运算 和 或非门 | **异或运算 和 异或门** | 同或运算 和 同或门

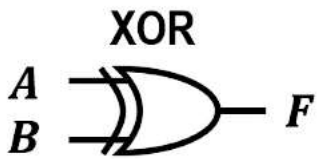
逻辑表达式

$$F = A \oplus B$$

真值表

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

电路符号



03 循环冗余校验

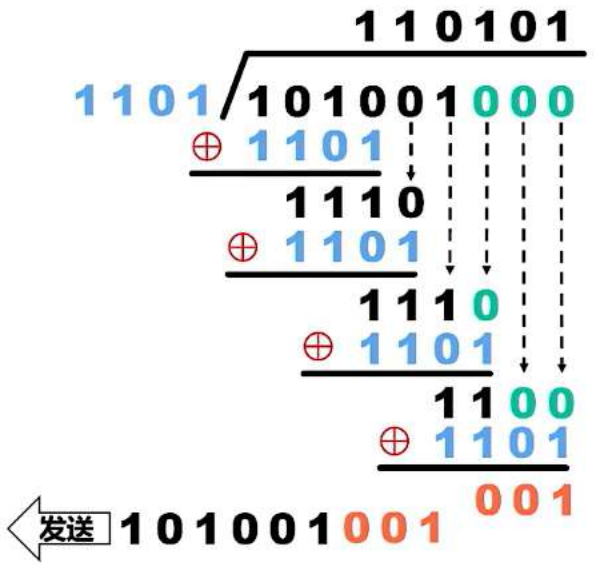
【CRC举例】待发送的数据为101001，生成多项式为$G(X) = X^3 + X^2 + 1$，计算冗余码。

1 构造“被除数”
待发送数据后面添加生成多项式最高次数个0

2 构造“除数”
生成多项式各项系数构成的比特串作为除数

3 做“二进制模2除法”
相当于对应位进行逻辑异或运算

4 检查“余数”
余数的位数应与生成多项式最高次数相同，如果位数不够，则在余数前补0来凑足位数。



复合逻辑运算与复合逻辑门

与非运算 和 与非门　　或非运算 和 或非门　　异或运算 和 异或门　　**同或运算 和 同或门**

异或运算 → 取反 → **同或运算**

逻辑表达式

异或运算：$F = A \oplus B$

同或运算：$F = A \odot B = \overline{A \oplus B}$



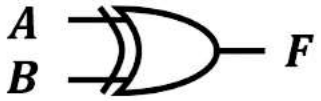
真值表

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

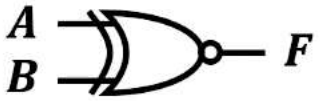
A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

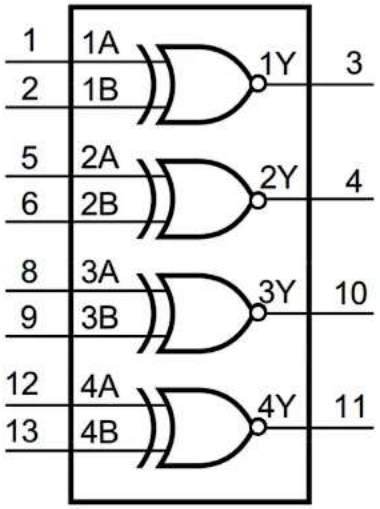
电路符号

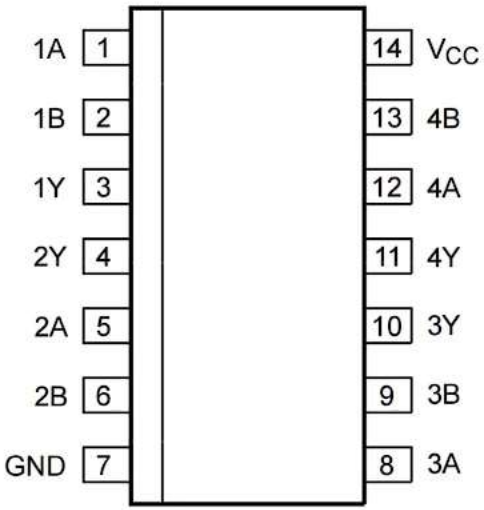
XOR



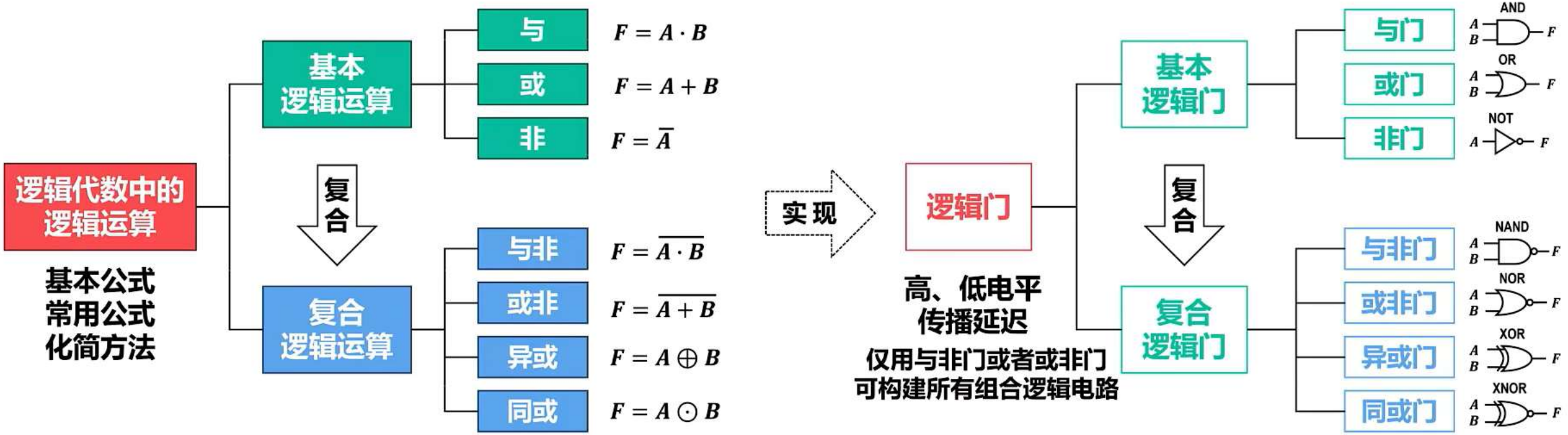
XNOR







逻辑代数和逻辑门



$F = A \cdot B$

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

$F = A + B$

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

$F = \overline{A}$

A	F
0	1
1	0

$F = \overline{A \cdot B}$

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0

$F = \overline{A + B}$

A	B	F
0	0	1
0	1	0
1	0	0
1	1	0

$F = A \oplus B$

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

$F = A \odot B$

A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

计算机408大纲变化对比

科目	2024大纲	2025大纲
数据结构		一、基本概念【新增】 (一)数据结构的基本概念 (二)算法的基本概念
	三、树与二叉树 (四)树与二叉树的应用 1.哈夫曼(Hufnan)树和哈夫曼编码 2.并查集及其应用	四、树与二叉树 (四)树与二叉树的应用 1.哈夫曼(Hufman)树和哈夫曼编码 2.并查集及其应用 3.堆及其应用【新 增 】
计算机组成原理	无变化	
操作系统	二、进程管理 (一)进程与线程 5.进程间通信 共享内存，消息传递，管道。 (二)CPU调度与上下文切换 4.典型调度算法【 删 除 】 先来先服务调度算法，短作业(短进程短线程)优先调度算法，时间片轮转调度算	二、进程管理 (一)进程与线程 5.进程间通信 共享内存，消息传递，管道，信号【新 增 】 (二)CPU调度与上下文切换 4.CPU调度算法【新增】 5.多处理机调度【新增】
	三、内存管理 (二)虚拟内存管理 3.页框分配	三、内存管理 (二)虚拟存储管理 3.页框分配与回收【 新 增 】
计算机网络	无变化	

2-1 数据表示的相关基本概念

